

claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_34671ac1-e8e\agent-a2842e77aec0e29c4.jsonl`  
- SHA-256: `6579a5c723128ec3c1ce21f88ad67676be8f41a3d279d3bd8113056b265e156e`  
- Source modified: `2026-06-21T10:22:00+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_34671ac1-e8e`  
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`
```

Transcript

1. user (2026-06-21T10:18:25.333Z)

Review the COMPUTE_DECOUPLED_SERVING **M5** change just committed on branch feat/serving-m5-truly-local-checks (HEAD). Inspect it with: `git diff origin/master...HEAD`, and read the full files:

- backend/config/startup.py (NEW ? the per-profile fail/warn-fast checks)
- backend/config/presets.py (added probe_cuda_available)
- backend/config/__init__.py (exports)
- backend/main.py (server main(): enforce after global_config load; sys.exit(1) on fatal)
- backend/worker/_main.py (_run_startup_checks helper; called in cmd_infer + cmd_train after _apply_overrides; returns rc 2 on fatal)
- tests/test_startup_checks.py (NEW, L1) + tests/test_profile_parity.py (NEW, L4)

The contract (must hold): DEFAULT-OFF ? when deployment.profile=="" NOTHING changes vs pre-M5 (byte-identical). Only an explicitly-selected profile triggers checks. cloud-serving + non-cloud storage = the ONLY fatal error (everything else is a warning) deliberately, to avoid false-positives (the CUDA probe is best-effort; ADC creds can't be confirmed offline). One isolated PR, default-OFF discipline. Report ONLY real, grounded issues (file:line). If a lens finds nothing real, return an empty findings array ? do not invent nits.

LENS = CORRECTNESS & DEFAULT-OFF SAFETY. Verify: (1) profile="" is a STRICT no-op everywhere (run_startup_checks returns [] early; enforce logs nothing, raises nothing) ? trace it. (2) Is enforce_startup_checks reachable in any path where it should NOT be, or skipped where it SHOULD run? In main.py it sits after the early-return CLI branches (setup/trim/debug) and before server+process ? is that the right placement? (3) Edge cases: a config with no 'deployment' section, no 'storage', no 'indexing.wasb' ? does getattr-chaining default safely without AttributeError? (4) The worker abort (return 2) vs main.py sys.exit(1) ? both correct? (5) Does _apply_overrides run BEFORE the check so a --set deployment.profile / storage.backend is seen? Confirm. Find any case where default behaviour changes or a check fires/raises incorrectly.

2. assistant (2026-06-21T10:18:27.786Z)

I'll review the M5 change with the CORRECTNESS & DEFAULT-OFF SAFETY lens. Let me start by examining the diff and the relevant files.

3. user (2026-06-21T10:18:31.634Z)

```
6f78acd feat(serving): M5 ? per-profile startup checks + L4 profile-parity test (default-OFF)  
fefa412 docs(serving): resolve §8 design picks (D16/D17) + reconcile per-video P3 vs M3 (#285)  
06ff4b7 test(api): lock gs://|gcs:// /api/process -> queryable API-DB video_id (B-P1 guard) (#284)
```

4. user (2026-06-21T10:18:31.911Z)

feat/serving-m5-truly-local-checks

```
6f78acd feat(serving): M5 ? per-profile startup checks + L4 profile-parity test (default-OFF)
```

5. assistant (2026-06-21T10:18:34.317Z)

Branch verified (HEAD is mine, only the M5 commit ahead of master). Now let me read the full diff and the key files.

6. user (2026-06-21T10:18:34.989Z)

```
backend/config/__init__.py      | 13 +-
backend/config/presets.py       | 12 ++
backend/config/startup.py       | 179 +++++
backend/main.py                 | 18 +-
backend/worker/__main__.py      | 23 +-
.../runlogs/RUNLOG_truly-local_TEMPLATE.md | 49 +++++
tests/test_profile_parity.py    | 139 +++++
tests/test_startup_checks.py    | 181 +++++
8 files changed, 611 insertions(+), 3 deletions(-)
```

7. user (2026-06-21T10:18:37.581Z)

```
1      """Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).
2
3      A deployment profile (`truly-local` / `cloud-serving` / `cpu-mock`) implies a
coherent
4      runtime. These checks fail/warn fast at process startup ? a loud signal *before* any
5      (expensive) job runs ? when the active profile is inconsistent with the environment:
6
7      * cloud-serving with a non-cloud storage backend ? a hard **error** (the one
config bug we
8      can detect with certainty + zero false-positive risk: a cloud profile that can't
reach a
9      bucket can never serve). This is the README §4.3 "loud error before any job".
10     * a **GPU mismatch** (e.g. `device='cuda'` on a box where CUDA isn't visible, or
`device='auto'`
11     falling back to CPU) ? a **warning**, never an error: the probe is best-effort
(torch may live
12     only in the detector subprocess env ? see `presets.probe_cuda_available`), and the
detector's
13     own healthcheck is the real authority. A warning surfaces it early without false-
failing.
14     * **creds that can't be confirmed** (no `GOOGLE_APPLICATION_CREDENTIALS` / not on
Cloud Run) ?
15     a **warning**: Application Default Credentials can come from the metadata server /
gcloud, which
16     we can't probe offline, so the storage backend's own init is the authority; we just
flag it.
17
18     **DEFAULT-OFF:** with no profile selected (`profile == ""`) this is a strict **no-op**
? exactly
19     the pre-M5 behaviour. Only an explicitly-selected profile triggers any check (mirrors
the loader,
20     which applies a preset only for an explicit profile). So nothing changes for today's
manual-knob
21     deployments.
22     """
23
24     from __future__ import annotations
25
26     import logging
27     import os
28     from dataclasses import dataclass
29     from typing import Any, Mapping, Optional
30
31     LEVEL_ERROR = "error"
32     LEVEL_WARN = "warn"
33
```

```

34     # Storage backends that count as "a cloud bucket" for cloud-serving coherence.
35     _CLOUD_STORAGE = ("gcs", "s3")
36     # Env signals that imply Google ADC *might* resolve without an explicit creds file.
37     _GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST",
38                       "CLOUDSDK_CONFIG", "GOOGLE_CLOUD_PROJECT")
39
40
41     @dataclass(frozen=True)
42     class StartupCheck:
43         """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-
44 up)."""
45         level: str
46         code: str      # stable, greppable code (e.g. "STORAGE_INCOHERENT")
47         message: str
48
49
50     class StartupCheckError(RuntimeError):
51         """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL
52 check."""
53
54     def __init__(self, failures: list[StartupCheck]):
55         self.failures = failures
56         super().__init__(
57             "deployment profile startup check failed: "
58             + "; ".join(f"[{c.code}] {c.message}" for c in failures)
59         )
60
61     def _creds_discoverable(env: Mapping[str, str]) -> bool:
62         """True if *some* Google ADC source is hinted by the environment. Best-effort ? a
63 False here
64         only downgrades to a WARNING, never a hard failure (the storage init is the
65 authority)."""
66         return any(env.get(k) for k in _GCP_ENV_HINTS)
67
68     def run_startup_checks(
69         config: Any,
70         *,
71         cuda_available: Optional[bool] = None,
72         env: Optional[Mapping[str, str]] = None,
73     ) -> list[StartupCheck]:
74         """Return the per-profile findings for ``config`` (empty when no profile is
75 selected).
76
77         ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ?
78 GPU checks
79 are skipped; the API server passes ``None``, the worker passes
80 ``presets.probe_cuda_available``).
81 Pure: no IO, no torch import ? the caller owns the probe so this stays importable
82 everywhere.
83 """
84         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
85         if not profile:
86             return [] # DEFAULT-OFF: no profile ? no checks (identical to pre-M5).
87
88         e = os.environ if env is None else env
89         storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
90         indexing = getattr(config, "indexing", None)
91         detector_impl = getattr(indexing, "detector_impl", "auto")
92         device = getattr(getattr(indexing, "wasb", None), "device", "cuda")
93
94         checks: list[StartupCheck] = []
95

```

```

91         if profile == "truly-local":
92             # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud
backend here is
93             # an inconsistent (but not fatal) choice worth surfacing.
94             if storage_backend not in ("local", "gdrive"):
95                 checks.append(StartupCheck(
96                     LEVEL_WARN, "STORAGE_UNUSUAL",
97                     f"profile 'truly-local' usually uses local/gdrive storage, but
storage_backend="
98                     f"'{storage_backend}'. Reads will go to a remote backend.",
99                 ))
100             # GPU heads-up (warning only ? the detector healthcheck is the authority; the
probe is
101             # best-effort and may be False even when a subprocess detector env has CUDA).
102             if cuda_available is False:
103                 if device == "cuda":
104                     checks.append(StartupCheck(
105                         LEVEL_WARN, "CUDA_NOT_VISIBLE",
106                         "device='cuda' but no CUDA GPU is visible to this process. If the
box truly "
107                         "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
108                         "'auto' for the CPU fallback (M4).",
109                     ))
110                 elif device == "auto":
111                     checks.append(StartupCheck(
112                         LEVEL_WARN, "CPU_FALLBACK",
113                         "device='auto' resolved to CPU (no CUDA GPU visible). Inference will
run on "
114                         "CPU ? correct, but much slower than a GPU.",
115                     ))
116
117         elif profile == "cloud-serving":
118             # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't
serve.
119             if storage_backend not in _CLOUD_STORAGE:
120                 checks.append(StartupCheck(
121                     LEVEL_ERROR, "STORAGE_INCOHERENT",
122                     f"profile 'cloud-serving' requires a cloud storage backend "
123                     f"'{'/'.join(_CLOUD_STORAGE)}', but storage_backend='{storage_backend}'."
124                     "A cloud job cannot read/write a local-only backend.",
125                 ))
126             elif storage_backend == "gcs" and not _creds_discoverable(e):
127                 checks.append(StartupCheck(
128                     LEVEL_WARN, "CREDS_UNCONFIRMED",
129                     "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying on
Application "
130                     "Default Credentials. The GCS backend will fail loudly at init if ADC
can't resolve.",
131                 ))
132             if device == "cpu":
133                 checks.append(StartupCheck(
134                     LEVEL_WARN, "CLOUD_CPU_DEVICE",
135                     "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
136                     "detection will not use the GPU.",
137                 ))
138             elif cuda_available is False and device in ("cuda", "auto"):
139                 checks.append(StartupCheck(
140                     LEVEL_WARN, "CLOUD_CUDA_NOT_VISIBLE",
141                     "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to
this process.",
142                 ))
143

```

```

144         elif profile == "cpu-mock":
145             # The mock profile is for CI/regression ? a real detector here defeats the
purpose.
146             if detector_impl != "stub":
147                 checks.append(StartupCheck(
148                     LEVEL_WARN, "MOCK_NONSTUB_DETECTOR",
149                     f"profile 'cpu-mock' normally uses detector_impl='stub', but
detector_impl="
150                     f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
151                     ))
152
153         return checks
154
155
156     def enforce_startup_checks(
157         config: Any,
158         *,
159         cuda_available: Optional[bool] = None,
160         env: Optional[Mapping[str, str]] = None,
161         raise_on_error: bool = True,
162         logger: Optional[logging.Logger] = None,
163     ) -> list[StartupCheck]:
164         """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
165         ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.
166
167         Returns the full findings list (so a caller can inspect/surface them). A no-op with
no logging
168         and no raise when no profile is selected (``run_startup_checks`` returns ``[]``)."""
169         log = logger or logging.getLogger(__name__)
170         checks = run_startup_checks(config, cuda_available=cuda_available, env=env)
171         for c in checks:
172             if c.level == LEVEL_ERROR:
173                 log.error("[STARTUP] %s: %s", c.code, c.message)
174             else:
175                 log.warning("[STARTUP] %s: %s", c.code, c.message)
176         errors = [c for c in checks if c.level == LEVEL_ERROR]
177         if errors and raise_on_error:
178             raise StartupCheckError(errors)
179         return checks
180

```

8. user (2026-06-21T10:18:38.000Z)

```

1         """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3         A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4         config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5         knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7         Precedence (lowest ? highest), realised by the loader::
8
9         built-in model defaults < profile preset < config.json < env < worker --set
10
11         This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12         module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13         (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14         profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15         today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16         spend is never entered silently).
17         """
18
19         from __future__ import annotations

```

```

20
21 import copy
22 import os
23 from typing import Any, Mapping, Optional
24
25 # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26 # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27 PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28 COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30 # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31 # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32 COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33     "truly-local": "local_gpu",
34     "cloud-serving": "cloud_run",
35     "cpu-mock": "cpu_mock",
36 }
37
38 # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39 # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40 # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41 # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42 # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43 PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44     "truly-local": {
45         "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46         # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47         # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48         "indexing": {"skip_ai_handoff": True},
49         "storage": {"backend": "local"},
50     },
51     "cloud-serving": {
52         "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53         # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54         "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55         "storage": {"backend": "gcs"},
56     },
57     "cpu-mock": {
58         "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59         # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61         "storage": {"backend": "local"},
62     },
63 }
64
65
66 def is_known_profile(profile: str) -> bool:
67     """True for a non-empty, recognised profile name."""
68     return bool(profile) and profile in PROFILE_PRESETS
69
70
71 def preset_for(profile: str) -> dict[str, Any]:
72     """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
73     module-level table), or ``{}`` for ``""`` / an unknown profile."""
74     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))

```

```

75
76
77 def _truthy(val: Optional[str]) -> bool:
78     """Common env-var truthiness (`1/true/yes/on`, case/space-insensitive)."""
79     if val is None:
80         return False
81     return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84 def _cuda_available() -> bool:
85     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86     ONLY when a caller opts into `probe_cuda`; any import/runtime failure means 'no
GPU'."""
87     try:
88         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89         # and optional). Only reached when a caller opts into probe_cuda.
90         import torch
91
92         return bool(torch.cuda.is_available())
93     except Exception:
94         return False
95
96
97 def probe_cuda_available() -> bool:
98     """Public best-effort CUDA probe (lazy torch import; `False` on any failure).
99
100     Exposed for the per-profile startup checks (`backend/config/startup.py`) so a
caller
101     that *wants* the GPU heads-up can pass the result in without the startup module
importing
102     torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
103     (native WASB), this returns `False` in the main process ? which is why the startup
checks
104     treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
105     error that could false-fail."""
106     return _cuda_available()
107
108
109 def suggest_profile(
110     env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
111 ) -> str:
112     """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
113
114     Signals, in priority order:
115
116     - `K_SERVICE` set (Cloud Run) ? `cloud-serving`
117     - `CI` truthy ? `cpu-mock`
118     - `probe_cuda` and a CUDA GPU present ? `truly-local`
119     - otherwise ? `` (no strong suggestion)
120
121     `probe_cuda` defaults to `False` so the common config-load path stays cheap and
never
122     imports torch; a CLI `--detect` or cloud-serving startup that wants the GPU probe
passes
123     `probe_cuda=True`. The result is advisory only: the caller decides whether to act
on it.
124
125     """
126     e = os.environ if env is None else env
127     if e.get("K_SERVICE"):
128         return "cloud-serving"
129     if _truthy(e.get("CI")):

```

```
129         return "cpu-mock"
130     if probe_cuda and _cuda_available():
131         return "truly-local"
132     return ""
133
```

9. assistant (2026-06-21T10:18:41.146Z)

Now let me read the integration points: `\_\_init\_\_.py`, `main.py`, and the worker.

10. user (2026-06-21T10:18:42.103Z)

```
1     """Configuration package.
2
3     Public API:
4         from backend.config import load_config, AppConfig, Config
5
6     This package is the canonical place for all configuration access.
7     """
8
9     from .loader import load_config, save_default_config
10    from .models import ( # re-export common ones
11        AppConfig,
12        Config,
13        DeploymentConfig,
14        IndexingConfig,
15        StitchingConfig,
16        StorageConfig,
17        ValidationConfig,
18    )
19    from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
20    from .startup import (
21        StartupCheck,
22        StartupCheckError,
23        enforce_startup_checks,
24        run_startup_checks,
25    )
26
27    __all__ = [
28        "load_config",
29        "save_default_config",
30        "AppConfig",
31        "Config",
32        "DeploymentConfig",
33        "IndexingConfig",
34        "StitchingConfig",
35        "StorageConfig",
36        "ValidationConfig",
37        "PROFILE_PRESETS",
38        "suggest_profile",
39        "probe_cuda_available",
40        "StartupCheck",
41        "StartupCheckError",
42        "enforce_startup_checks",
43        "run_startup_checks",
44    ]
45
```

11. user (2026-06-21T10:18:42.590Z)

```
1     import os
2     import sys
3     import argparse
4     import logging
```



```

5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    load_dotenv()
11
12    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
13    logging.basicConfig(
14        level=logging.INFO,
15        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
16        datefmt='%H:%M:%S'
17    )
18    logger = logging.getLogger(__name__)
19    from fastapi import FastAPI, HTTPException
20    from fastapi.middleware.cors import CORSMiddleware
21    from pydantic import BaseModel
22    from typing import List, Dict, Any, Optional
23
24    from fastapi.staticfiles import StaticFiles
25    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26    from backend.pipeline.validators.base import registry
27    from backend.infrastructure.database import Database
28    from backend.pipeline.segmenters.base import segmenter_registry
29    from backend.pipeline.stitcher.compiler import VideoCompiler
30    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32    from backend.config import ( # new typed config
33        load_config as load_app_config,
34        AppConfig,
35        StitchingConfig,
36        enforce_startup_checks,
37        StartupCheckError,
38    )
39    from backend.results import Failure # standardized error/result contract
40    from backend.reporting import emit_indexer_report
41    from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
42
43    app = FastAPI(title="Sports Highlight Indexer API")
44
45    # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is
46    # rejected by browsers per the fetch spec and is an unsafe default to carry into the
47    # auth/tenancy work; this tool uses no cookies, so credentials stay off.
48    app.add_middleware(
49        CORSMiddleware,
50        allow_origins=["*"],
51        allow_credentials=False,
52        allow_methods=["*"],
53        allow_headers=["*"],
54    )
55
56    # Serves static frontend files directly (now under api/ per approved structure)
57    os.makedirs("backend/api/static", exist_ok=True)
58    app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
59
60    @app.get("/", response_class=HTMLResponse)
61    def serve_index():
62        index_path = "backend/api/static/index.html"
63        if os.path.exists(index_path):
64            with open(index_path, "r", encoding="utf-8") as f:
65                return f.read()

```

```

66         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
67
68     # Global variables initialized at startup
69     db_instance: Optional[Database] = None
70     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
71
72     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
73     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
74     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
75     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
76     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
77     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
78     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
79     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
80         JobWaiting,
81         _arm_wake_timer,
82         _drain_due_jobs,
83         _get_executor,
84         _job_to_request,
85         _MAX_DRAIN_WAKE_SEC,
86         _run_claimed_job,
87         _schedule_next_drain_if_waiting,
88     )
89
90     # Legacy thin wrapper for any remaining direct calls during transition.
91     # New code should import load_app_config / AppConfig from backend.config directly.
92     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
93         cfg = load_app_config(config_path)
94         return cfg.model_dump(mode="json")
95
96     # --- API Models ---
97     class ProcessRequest(BaseModel):
98         video_path: str
99         player_pool: Optional[List[str]] = None
100         max_frames: Optional[int] = None
101         segmenter: Optional[str] = None
102
103     class QueryRequest(BaseModel):
104         video_id: str
105         server: Optional[str] = None
106         receiver: Optional[str] = None
107         duration_min: Optional[float] = None
108         event_type: Optional[str] = None
109
110     class CompileRequest(BaseModel):
111         video_id: str
112         segment_ids: List[int]
113         output_filename: str
114
115     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
116         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
117
118         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
119         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
120         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
121         """
122         if segments:
123             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
124         else:

```

```

125         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
126
127
128     # --- API Endpoints ---
129     @app.get("/api/config")
130     def get_config():
131         return global_config
132
133     @app.get("/api/health")
134     def health():
135         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
136         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
137         confirm the engine is reachable and which detector is wired. Never raises."""
138         sport = getattr(global_config, "sport", None)
139         indexing = getattr(global_config, "indexing", None)
140         return {
141             "status": "ok",
142             "sport": sport,
143             "default_segmenter": getattr(indexing, "default_segmenter", None),
144         }
145
146     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
147         """The full hash ? validate ? segment ? report pipeline for one video.
148
149         This is the former synchronous /api/process body, unchanged in behavior, now run on
150         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
151         the sync endpoint used to return (UI compatibility); the per-video observability
152         report is still emitted in `finally:` and grafted as response["reports"].
153
154         `progress` is an optional callable(stage: str) used to surface coarse job progress.
155         Raises on unexpected internal errors ? the caller records those as a job failure
156         (after this function has already restored the pre-run segments, see below).
157         """
158         notify = progress or (lambda stage: None)
159
160         notify("hashing")
161         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
162         (identity ?
163         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
164         original
165         # req.video_path (the source URI) is kept for the DB record + report; only the file-
166         reading
167         # consumers (hash / validators / segmenter) use the resolved local path.
168         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
169         "storage", None))
170         video_id = compute_video_id(local_video)
171         was_reingest = db_instance.get_video(video_id) is not None
172
173         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
174         report)
175         notify("validating")
176         logger.info(f"Running validations for {req.video_path}...")
177         val_results, val_context = registry.run_all_with_context(local_video, global_config)
178         failures = [r for r in val_results if not r.passed]
179
180         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
181         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
182         "motion")
183         seg_name = req.segmenter or default_seg
184         segmenter_actual = None
185         segmentation_ran = False
186         response: Optional[Dict[str, Any]] = None
187         try:
188             if failures:
189                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates

```

```

184         # status; it no longer destroys the video's prior good segments.
185         db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
186         response = {
187             "video_id": video_id,
188             "status": "failed",
189             "message": "Video failed validation checks.",
190             "results": [r.model_dump() for r in val_results],
191         }
192         return response
193
194     # 2. Run segmenter & indexer
195     logger.info("[API] Video passed checks. Segmenting and indexing...")
196     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
197
198     segmenter_cls = segmenter_registry.get(seg_name)
199     if not segmenter_cls:
200         # Submit-time validation already 400s unknown names; this is the defensive
201         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
202         available = list(segmenter_registry.list_available().keys())
203         response = {
204             "video_id": video_id,
205             "status": "failed",
206             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
207             "results": [r.model_dump() for r in val_results],
208             "play_segments": [],
209         }
210         return response
211
212     logger.info(f"[API] Using segmenter: {seg_name}")
213     notify(f"segmenting ({seg_name})")
214     segmenter_actual = seg_name
215     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
216     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
217     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
218     segmenter = segmenter_cls(db_instance, global_config)
219     try:
220         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
221     except Exception:
222         # A raising segmenter must not leave half-written rows: restore the pre-run
223         # state (same destroy-after-confirm contract as the Failure branch), then
let
224         # the job worker record the failure.
225         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
226         raise
227     segmentation_ran = True
228
229     if isinstance(result, Failure):
230         logger.error(f"[API] Segmenter failed: {result.message}")
231         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
232         response = {
233             "video_id": video_id,
234             "status": "failed",
235             "message": f"Segmenter '{seg_name}' failed: {result.message}",
236             "results": [r.model_dump() for r in val_results],
237             "play_segments": [],
238         }
239         return response
240

```

```

241         segments = result.value if hasattr(result, "value") else result
242         notify("finalizing")
243         _finalize_reingest(video_id, segments, play_wm, cand_wm)
244
245         response = {
246             "video_id": video_id,
247             "status": "success",
248             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
249             "results": [r.model_dump() for r in val_results],
250             "play_segments": segments,
251         }
252         return response
253     finally:
254         # Always emit the per-video observability report (next to the video, or to
255         # report.output_dir). Never raises. Surface the paths in the response.
256         paths = emit_indexer_report(
257             db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
258             val_results=val_results, val_context=val_context,
259             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
260             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
261         )
262         if paths and isinstance(response, dict):
263             response["reports"] = paths
264
265
266     @app.post("/api/process", status_code=202)
267     def process_video(req: ProcessRequest):
268         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
269
270         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
271         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
272         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
273         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
274         """
275         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
276         try:
277             validate_input_path(req.video_path, allowed_root=allowed_root)
278             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
279             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
280             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
281         except ValueError as e:
282             raise HTTPException(status_code=400, detail=str(e)) from e
283         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
284         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
285         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
286         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
287         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
288         # rejects a non-local URI as out-of-root.
289         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
290             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
291         if req.segmenter and not segmenter_registry.get(req.segmenter):
292             available = list(segmenter_registry.list_available().keys())
293             raise HTTPException(
294                 status_code=400,

```

```

295         detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
296     )
297
298     job_id = uuid.uuid4().hex
299     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
300                                 req.player_pool, req.max_frames):
301         raise HTTPException(status_code=500, detail="Failed to persist job record.")
302     _get_executor().submit(_drain_due_jobs)
303     return JSONResponse(
304         status_code=202,
305         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
306     )
307
308
309 @app.get("/api/jobs/{job_id}")
310 def get_job(job_id: str):
311     """Job state: {status, progress, result, reports}. `status` = execution state
312     (queued|running|done|failed); the pipeline verdict is result["status"]."""
313     job = db_instance.get_job(job_id)
314     if not job:
315         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
316     result = job.get("result")
317     return {
318         "job_id": job["id"],
319         "status": job["status"],
320         "progress": job.get("progress"),
321         "video_id": job.get("video_id"),
322         "error": job.get("error"),
323         "result": result,
324         "reports": (result or {}).get("reports"),
325         "created_at": job.get("created_at"),
326         "started_at": job.get("started_at"),
327         "finished_at": job.get("finished_at"),
328     }
329
330 # --- Chunked upload (B2, PR-2) -----
331 # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
332 # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
333 # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
334 # sequential-part logic, and abort cleanup ? is unchanged.
335 from backend.api import uploads as uploads_api
336
337 app.include_router(uploads_api.router)
338
339
340 # --- Highlight download (B3, PR-2) -----
341
342 @app.get("/api/highlights/{video_id}/{filename}")
343 def download_highlight(video_id: str, filename: str):
344     """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
345     (which only writes into output_dir; the browser previously got back an unreachable
346     server-local path)."""
347     try:
348         name = safe_output_filename(filename)
349     except ValueError as e:
350         raise HTTPException(status_code=400, detail=str(e)) from e
351     if not db_instance.get_video(video_id):
352         raise HTTPException(status_code=404, detail="Indexed video record not found.")
353     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
354     path = os.path.join(output_dir, name)

```

```

355         try:
356             path = resolve_under_root(path, output_dir)
357         except ValueError as e:
358             raise HTTPException(status_code=400, detail=str(e)) from e
359     if not os.path.isfile(path):
360         raise HTTPException(status_code=404,
361                             detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
362     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
363     return FileResponse(path, media_type=media, filename=name)
364
365
366 @app.post("/api/query")
367 def query_play_segments(req: QueryRequest):
368     filters = {
369         "server": req.server,
370         "receiver": req.receiver,
371         "duration_min": req.duration_min,
372         "event_type": req.event_type
373     }
374     segments = db_instance.query_play_segments(req.video_id, filters)
375     return {"play_segments": segments}
376
377 @app.post("/api/compile")
378 def compile_highlights(req: CompileRequest):
379     video = db_instance.get_video(req.video_id)
380     if not video:
381         raise HTTPException(status_code=404, detail="Indexed video record not found.")
382
383     # Retrieve matching play segments from db
384     all_segments = db_instance.query_play_segments(req.video_id, {})
385     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
386
387     if not matched_segments:
388         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
389
390     # Reject path traversal / absolute output names (os.path.join would otherwise let an
391     # absolute or '..'-laden output_filename write anywhere on disk).
392     try:
393         out_name = safe_output_filename(req.output_filename)
394     except ValueError as e:
395         raise HTTPException(status_code=400, detail=str(e)) from e
396
397     # The configured shot-count floor (stitching.min_shot_count) applies on the API
398     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
399     # metadata are exempt: the floor filters known counts only, so explicitly
400     # requested manual/legacy segments can't silently vanish under the default floor.
401     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
402     kept = []
403     for s in matched_segments:
404         meta = s.get("metadata") or {}
405         sc = meta.get("shot_count") if isinstance(meta, dict) else None
406         try:
407             if sc is not None and int(sc) < stitching.min_shot_count:
408                 continue
409             except (TypeError, ValueError):
410                 pass # unparseable count -> treat as unknown, keep
411             kept.append(s)
412     if len(kept) < len(matched_segments):
413         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
414                     f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")

```

```

415         if not kept:
416             raise HTTPException(status_code=400,
417                                 detail=f"All {len(matched_segments)} matching segments fall
below "
418
419         f"stitching.min_shot_count={stitching.min_shot_count}.")
420     # Extract start/end time pairs
421     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
422
423     # Run compiler with the configured stitching paddings + optional branding watermark
424     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
425     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
426                             end_padding=stitching.end_padding,
watermark=stitching.watermark)
427     try:
428         out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
429         # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
430         # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).
431         download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
432         return {"status": "success", "filepath": out_path, "download_url": download_url}
433     except Exception as e:
434         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
435
436     # --- CLI Debug Utility & Orchestration ---
437     def run_cli_diagnose_clip(video_path: str, timestamp: float):
438         """CLI debugging utility to examine segment properties around a timestamp."""
439         print("=" * 60)
440         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
441         print("=" * 60)
442
443         cfg = load_config() # legacy wrapper still works, returns dict for now
444
445         # 1. Validation check diagnostics
446         print("[DIAGNOSTIC] Running validation framework...")
447         results = registry.run_all(video_path, cfg)
448         for r in results:
449             status_symbol = "[PASS]" if r.passed else "[FAIL]"
450             print(f" {status_symbol} {r.validator_name}: {r.message}")
451             if r.details:
452                 print(f"      Details: {r.details}")
453
454         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
455         print("-" * 60)
456         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
457         import cv2
458         cap = cv2.VideoCapture(video_path)
459         if not cap.isOpened():
460             print("[FAIL] OpenCV could not open video.")
461             return
462
463         fps = cap.get(cv2.CAP_PROP_FPS)
464         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
465
466         start_frame = max(0, int((timestamp - 3.0) * fps))
467         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
468
469         # Measure frame-by-frame changes in sample region
470         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
471         prev_gray = None
472         motions = []

```



```

473
474     for _ in range(start_frame, end_frame):
475         ret, frame = cap.read()
476         if not ret:
477             break
478         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
479         gray = cv2.GaussianBlur(gray, (21, 21), 0)
480
481         if prev_gray is not None:
482             diff = cv2.absdiff(prev_gray, gray)
483             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
484             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
485             motions.append(motion_ratio)
486             prev_gray = gray
487
488     cap.release()
489
490     if motions:
491         avg_motion = sum(motions) / len(motions)
492         # Support both legacy dict (from wrapper) and future direct model
493         if hasattr(cfg, "indexing"):
494             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
495         else:
496             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
497         print(f" Sample Frame Count: {len(motions)}")
498         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
499         if avg_motion > threshold:
500             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
501         else:
502             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
503     else:
504         print(" [ERROR] No visual frames were extracted in the sample range.")
505
506     print("=" * 60)
507
508     def main():
509         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
510         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
511         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
512         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
513         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
514         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
515         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
516         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
517
518         available_segmenters = list(segmenter_registry.list_available().keys())
519         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
520         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
521         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
522         parser.add_argument("--trim-output", help="Output filename for the debug trimmed

```

```

segment (saves in output-dir unless absolute path)")
523
524     args = parser.parse_args()
525
526     if args.setup:
527         # Invoke the diagnostics/bootstrap script (renamed from the old root
528         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
529         # build system). Resolve its path from this file so `indexer --setup`
530         # works regardless of the caller's cwd.
531         import subprocess
532         doctor_path = os.path.join(
533             os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
534             "scripts", "doctor.py",
535         )
536         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
537         subprocess.run([sys.executable, doctor_path])
538         return
539
540     if args.trim_start is not None and args.trim_end is not None:
541         if not args.video_path:
542             print("[ERROR] Please provide --video-path to extract a segment.")
543             return
544
545         # Determine output path
546         out_filename = args.trim_output or "trimmed_segment.mp4"
547         if os.path.isabs(out_filename):
548             out_path = out_filename
549             out_dir = os.path.dirname(out_path)
550             out_name = os.path.basename(out_path)
551         else:
552             out_dir = args.output_dir
553             out_path = os.path.join(out_dir, out_filename)
554             out_name = out_filename
555
556         os.makedirs(out_dir, exist_ok=True)
557         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
558
559         # global_config isn't initialized this early; load the typed config so the trim
560         # clip carries the same configured stitching paddings + watermark as a real
561         # highlight (watermark default-on).
562         stitching = load_app_config().stitching
563         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
564                                 end_padding=stitching.end_padding,
565                                 watermark=stitching.watermark)
566         try:
567             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
568             args.trim_end)], out_name)
569             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
570         except Exception as e:
571             print(f"[ERROR] Failed to compile trimmed segment: {e}")
572             return
573
574     if args.debug_clip is not None:
575         if not args.video_path:
576             print("[ERROR] Please provide --video-path to debug a specific clip.")
577             return
578         run_cli_diagnose_clip(args.video_path, args.debug_clip)
579         return
580
581     # Initialize Global Config and DB
582     global global_config, db_instance
583     global_config = load_app_config() # returns typed AppConfig
584
585     # M5: per-profile startup checks ? fail fast on an incoherent ACTIVE profile (e.g. a

```

```

584         # cloud-serving profile pointed at local-only storage) BEFORE any work. A strict no-
op when
585         # no deployment.profile is selected (the default), so today's behaviour is
unchanged. No GPU
586         # probe here (the server stays torch-light; the detector healthcheck is the GPU
authority).
587         try:
588             enforce_startup_checks(global_config, logger=logger)
589         except StartupCheckError as e:
590             logger.error("[STARTUP] refusing to start: %s", e)
591             sys.exit(1)
592
593         # The CLI --output-dir overrides the configured output directory. It now lives
594         # on the typed model (OutputConfig.output_dir), so every consumer ? including
595         # /api/compile ? reads the same value instead of a write-only runtime hack.
596         global_config.output.output_dir = args.output_dir
597         os.makedirs(global_config.output.output_dir, exist_ok=True)
598
599         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
600         db_instance = Database(db_path)
601
602         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
603         # the executor is in-process. Mark them failed so pollers see a terminal state.
604         swept = db_instance.fail_stale_jobs()
605         if swept:
606             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
607
608         # CLI processing mode (no web UI needed)
609         if args.video_path and not args.server:
610             print(f"[CLI] Processing video file: {args.video_path}")
611             # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
612             # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
613             # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
614             storage_cfg = getattr(global_config, "storage", None)
615             try:
616                 input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
617             except ValueError as e:
618                 print(f"[FAIL] {e}")
619                 return
620             if input_ref.backend == "local" and not os.path.exists(input_ref.key):
621                 print(f"[FAIL] Video file not found: {args.video_path}")
622                 return
623             local_video = storage.acquire_local_input(args.video_path, storage_cfg)
624
625             video_id = compute_video_id(local_video)
626             was_reingest = db_instance.get_video(video_id) is not None
627
628             # Validate (with-context variant surfaces ffmpeg_meta for the report)
629             print(f"[CLI] Validating...")
630             val_results, val_context = registry.run_all_with_context(local_video,
global_config)
631             failures = [r for r in val_results if not r.passed]
632
633             # Save video status
634             status = "failed" if failures else "processed"
635             db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
636
637             seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")

```

```

638         segmenter_actual = None
639         segmentation_ran = False
640         try:
641             print("\n" + "="*40)
642             print("VALIDATION SUMMARY")
643             print("="*40)
644             for r in val_results:
645                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
646                 print(f"{status_symbol} {r.validator_name}: {r.message}")
647             print("="*40 + "\n")
648
649             if failures:
650                 print("[FAIL] Video failed checks. Indexing halted.")
651                 return
652
653             # Index
654             segmenter_cls = segmenter_registry.get(seg_name)
655             if not segmenter_cls:
656                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
657                 return
658
659             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
660             segmenter_actual = seg_name
661             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
662             segmenter = segmenter_cls(db_instance, global_config)
663             result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
664             segmentation_ran = True
665
666             if isinstance(result, Failure):
667                 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
668                 print("[FAIL] Indexing halted for video.")
669                 segments = []
670             else:
671                 segments = result.value if hasattr(result, "value") else result
672                 print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
673                 # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
674                 _finalize_reingest(video_id, segments, play_wm, cand_wm)
675             finally:
676                 # Always emit the per-video observability report. Never raises.
677                 paths = emit_indexer_report(
678                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
679                     val_results=val_results, val_context=val_context,
680                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
681                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
682                 )
683             if paths:
684                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
685                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
686                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
687             return
688
689         # Start Local Server Mode
690         if args.server or not args.video_path:
691             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
692             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
693             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
694             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
695             _get_executor().submit(_drain_due_jobs)

```

```

696         uvicorn.run(app, host="127.0.0.1", port=args.port)
697
698     if __name__ == "__main__":
699         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
700         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
701         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
702         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
703         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
704         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
705         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
706         import backend.main as _bm
707
708         _bm.main()
709

```

12. assistant (2026-06-21T10:18:44.941Z)

Now let me read the worker and the tests.

13. user (2026-06-21T10:18:45.487Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14    (validator registry ? segmenter ? report), never a forked pipeline.
15    """
16    from __future__ import annotations
17
18    import argparse
19    import csv
20    import json
21    import logging
22    import os
23    import sys
24    import tempfile
25    from typing import Any, List
26
27    from backend import storage
28    from backend.config import (
29        StartupCheckError,
30        enforce_startup_checks,
31        load_config,
32        probe_cuda_available,
33    )
34    from backend.infrastructure.database import Database
35    from backend.pipeline.segmenters.base import segmenter_registry

```

```

36     from backend.pipeline.validators.base import registry as validator_registry
37     from backend.reporting import emit_indexer_report
38     from backend.results import Failure
39     from backend.utils.hashing import compute_video_id
40
41     logger = logging.getLogger("backend.worker")
42
43
44     def _coerce(v: str) -> Any:
45         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
46         str."""
47         low = v.lower()
48         if low in ("true", "false"):
49             return low == "true"
50         for cast in (int, float):
51             try:
52                 return cast(v)
53             except ValueError:
54                 pass
55         return v
56
57     def _apply_overrides(config: Any, sets: List[str]) -> None:
58         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
59         indexing.skip_ai_handoff=true)."""
60         for kv in sets or []:
61             key, sep, val = kv.partition("=")
62             if not sep:
63                 raise ValueError(f"--set expects k=v, got {kv!r}")
64             parts = key.split(".")
65             obj = config
66             for p in parts[:-1]:
67                 obj = getattr(obj, p)
68             setattr(obj, parts[-1], _coerce(val))
69
70     def _write_intervals_csv(segments: List[dict], path: str) -> None:
71         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
72         friendly
73         artifact (same schema as the rally-annotator labels)."""
74         with open(path, "w", newline="") as f:
75             w = csv.writer(f)
76             w.writerow(["start", "end", "shot_count", "ending_reason"])
77             for s in segments:
78                 w.writerow([
79                     f'{float(s.get("start_time", 0.0)):.3f}',
80                     f'{float(s.get("end_time", 0.0)):.3f}',
81                     s.get("shot_count", ""),
82                     s.get("ending_reason", s.get("shot_count_confidence", "")),
83                 ])
84
85     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
86     None:
87         with open(path, "w") as f:
88             json.dump({
89                 "video_id": video_id,
90                 "status": status,
91                 "segment_count": len(segments),
92                 "segments": [{"start": float(s.get("start_time", 0.0)),
93                             "end": float(s.get("end_time", 0.0))} for s in segments],
94             }, f, indent=2)
95
96     def _run_startup_checks(config) -> bool:

```

```

97         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
98         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
99         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A no-op
(return
100         True) when no deployment.profile is selected."""
101         try:
102             enforce_startup_checks(config, cuda_available=probe_cuda_available(),
logger=logger)
103             return True
104         except StartupCheckError:
105             return False # already logged at ERROR by enforce_startup_checks
106
107
108     def cmd_infer(args: argparse.Namespace) -> int:
109         config = load_config(args.config) if args.config else load_config()
110         _apply_overrides(config, args.set)
111         if not _run_startup_checks(config):
112             return 2
113         storage_cfg = config.storage.model_dump()
114
115         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
116         os.makedirs(scratch, exist_ok=True)
117         config.output.output_dir = scratch
118
119         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
120         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
121         print(f"[worker] pulled {args.video_uri} -> {local_video}")
122
123         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
124         video_id = compute_video_id(local_video)
125
126         # 3. VALIDATE (same registry as the main CLI).
127         val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
128         failures = [r for r in val_results if not r.passed]
129         db = Database(os.path.join(scratch, config.output.db_name))
130         db.add_video(video_id, local_video, "failed" if failures else "processed",
[r.model_dump() for r in val_results])
131
132
133         segments: List[dict] = []
134         segmenter_actual = None
135         segmentation_ran = False
136         status = "failed"
137         try:
138             if failures:
139                 print("[worker] validation FAILED: "
+ "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
140                 return 2
141             seg_name = args.segmenter or config.indexing.default_segmenter
142             segmenter_cls = segmenter_registry.get(seg_name)
143             if not segmenter_cls:
144                 print(f"[worker] unknown segmenter: {seg_name!r} "
f"(have {list(segmenter_registry.list_available())})")
145                 return 2
146             segmenter_actual = seg_name
147             result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
148             segmentation_ran = True
149             if isinstance(result, Failure):
150                 print(f"[worker] segmenter FAILED: {result.message}")

```

```

153         return 3
154     segments = result.value if hasattr(result, "value") else result
155     status = "success"
156     # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
157     # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
158     # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
159     if not segments:
160         detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
161                         or getattr(config.indexing, "detector_impl", None) or
"auto")
162         logger.warning(
163             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
164             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
165             "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
166             "the WASB env produced no detections, or the input resolution differs
from the tuned "
167             "proxy resolution. Re-run with -v for the native command + frame
counts.",
168             video_id, segmenter_actual, detector_impl,
169         )
170     finally:
171         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
172         emit_indexer_report(
173             db=db, video_id=video_id, video_path=local_video, config=config,
174             val_results=val_results, val_context=val_context,
175             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
176             was_reingest=False, segmentation_ran=segmentation_ran,
177         )
178
179     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
180     out_local = os.path.join(scratch, "outputs", video_id)
181     os.makedirs(out_local, exist_ok=True)
182     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
183     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
184
185     out_prefix = args.out_uri.rstrip("/")
186     pushed = []
187     for fn in sorted(os.listdir(out_local)):
188         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
189         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
190         pushed.append(uri)
191
192     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
193     for u in pushed:
194         print("    ", u)
195     return 0
196
197
198     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
199     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
200     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
201
202
203     def _probe_video_fps_width(path: str):

```



```

204     """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
205
206     The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
207     fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
208     cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
209     inference report (it flags the fallback) but corrupting for REAL training data. So
here we
210     probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
211     rather than guesses.
212     """
213     import json
214     import subprocess
215     try:
216         out = subprocess.check_output(
217             ["ffprobe", "-v", "error", "-select_streams", "v:0",
218              "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
219             stderr=subprocess.DEVNULL).decode()
220         st = (json.loads(out).get("streams") or [{}])[0]
221         num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
222         fps = float(num) / float(den or "1")
223         width = float(st.get("width") or 0)
224         if fps > 0 and width > 0:
225             return fps, width
226     except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
227         pass
228     return None
229
230
231 def _resolve_train_detector(args: argparse.Namespace, config: Any):
232     """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
233     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
234     stub=CPU mock). Returns the runner, or prints an error + returns None.
235
236     Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the
237     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
238     no shuttle detector and is rejected.
239     """
240     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
241
242     seg_cls = segmenter_registry.get(args.segmenter)
243     if not seg_cls:
244         print(f"[worker] unknown segmenter: {args.segmenter!r} "
245               f"(have {list(segmenter_registry.list_available())})")
246         return None
247     seg = seg_cls(None, config)
248     if not isinstance(seg, TrajectoryHybridSegmenter):
249         print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
250               f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
251     return None
252     try:
253         runner, _ = seg._resolve_runner(config.get("indexing", {}))
254     except NotImplementedError as e:
255         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
256         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
257         print(f"[worker] detector not available: {e}")

```

```

258         return None
259     ok, msg = runner.healthcheck()
260     if not ok:
261         print(f"[worker] detector environment not ready: {msg}")
262         return None
263     print(f"[worker] detector ready ({args.segmenter}): {msg}")
264     return runner
265
266
267     def cmd_train(args: argparse.Namespace) -> int:
268         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
269         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
270
271         Two trajectory sources (the train pipeline + harness are identical either way):
272         * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
273         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
274         * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
275         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
276         is the M3.5 "first real training" path; only the trajectory source flips.
277         """
278         import subprocess
279
280         config = load_config(args.config) if args.config else load_config()
281         _apply_overrides(config, args.set)
282         if not _run_startup_checks(config):
283             return 2
284         storage_cfg = config.storage.model_dump()
285
286         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
287         labels_dir = os.path.join(scratch, "labels")
288         traj_dir = os.path.join(scratch, "trajectories")
289         videos_dir = os.path.join(scratch, "videos")
290         os.makedirs(labels_dir, exist_ok=True)
291         os.makedirs(traj_dir, exist_ok=True)
292
293         corpus = args.corpus_uri.rstrip("/")
294         label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
295                             if u.lower().endswith(".csv"))
296         if len(label_uris) < 2:
297             print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
298                   f"under {corpus}/labels/")
299             return 2
300
301         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
302         runner = None
303         video_by_stem: dict = {}
304         if args.trajectory_source == "detector":
305             os.makedirs(videos_dir, exist_ok=True)
306             runner = _resolve_train_detector(args, config)
307             if runner is None:
308                 return 2
309             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
310                 vname = storage.parse_uri(vuri).name
311                 if not vname.lower().endswith(_VIDEO_EXTS):
312                     continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
313                 video_by_stem[os.path.splitext(vname)[0]] = vuri

```

```

314
315     print(f"[worker] trajectory source: {args.trajectory_source}")
316     specs: List[dict] = []
317     for uri in label_uris:
318         fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
319         name = fname.replace(".rallies.csv", "").replace(".csv", "")
320         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
321         if args.trajectory_source == "detector":
322             vuri = video_by_stem.get(name)
323             if not vuri:
324                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
325                 continue
326                 local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
327                                     config=storage_cfg)
328                 video_id = compute_video_id(local_video)
329                 # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
330                 # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
331                 meta = _probe_video_fps_width(local_video)
332                 if meta is None:
333                     print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
334                     continue
335                     fps, frame_width = meta
336                     traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
337                     if not traj:
338                         print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
339                         continue
340                         specs.append({"name": name, "trajectory": traj, "labels": local_label,
341                                     "fps": fps, "frame_width": frame_width})
342                     else:
343                         from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
344                         traj = os.path.join(traj_dir, f"{name}_traj.csv")
345                         synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
346                         specs.append({"name": name, "trajectory": traj, "labels": local_label,
347                                     "fps": args.fps, "frame_width": args.frame_width})
348
349         if len(specs) < 2:
350             print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
351             return 2
352         manifest_path = os.path.join(scratch, "manifest.json")
353         with open(manifest_path, "w", encoding="utf-8") as f:
354             json.dump(specs, f, indent=2)
355         src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
356         print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
357
358         out_dir = os.path.join(scratch, "artifacts")
359         cmd = [sys.executable, "-m", "training.gen0.harness",
360               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
361         if args.floor:
362             floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
363                           if "://" in args.floor else args.floor)
364             cmd += ["--floor", floor_local]
365         print(f"[worker] training:", " ".join(cmd))

```

```

366     rc = subprocess.run(cmd).returncode
367     if rc != 0:
368         print(f"[worker] gen0 harness failed (rc={rc})")
369         return rc
370
371     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
372     bundle = os.path.join(out_dir, args.version)
373     out_prefix = args.out_uri.rstrip("/")
374     pushed = []
375     for fn in sorted(os.listdir(bundle)):
376         uri = f"{out_prefix}/weights/{args.version}/{fn}"
377         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
378         pushed.append(uri)
379     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
380     for u in pushed:
381         print("    ", u)
382     return 0
383
384
385     def build_parser() -> argparse.ArgumentParser:
386         p = argparse.ArgumentParser(prog="python -m backend.worker",
387                                     description="Storage-aware worker: pull ? run pipeline ?
push.")
388         p.add_argument("-v", "--verbose", action="store_true",
389                       help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
390         sub = p.add_subparsers(dest="cmd", required=True)
391
392         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
393         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
394         pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
395         pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
396         pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
397         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
398         pi.add_argument("--max-frames", type=int, default=None)
399         pi.add_argument("--set", action="append", default=[], metavar="k=v",
400                       help="config override, e.g. --set indexing.skip_ai_handoff=true")
401         pi.set_defaults(func=cmd_infer)
402
403         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
404         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
405                       "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
406         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
407         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
408         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
409                       default="synth",
410                       help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
411                       "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
412                       "(needs videos/ in the corpus; honours detector_impl).")
412         pt.add_argument("--segmenter", default="wasb_hybrid",
413                       help="trajectory hybrid whose detector is used for --trajectory-
source detector")
414         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
415         pt.add_argument("--config", default=None)

```

```

416         pt.add_argument("--scratch", default=None)
417         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
418         pt.add_argument("--frame-width", type=float, default=1280.0,
419             help="synth trajectory frame width (detector probes the video)")
420         pt.add_argument("--set", action="append", default=[], metavar="k=v")
421         pt.set_defaults(func=cmd_train)
422         return p
423
424
425     def _setup_logging(verbose: bool) -> None:
426         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
427         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
428         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
429         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
430         entry), not at import, so importing the module never reconfigures a caller's
logging."""
431         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
432         # configured logging at import), which would leave the level at WARNING and re-
suppress the
433         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
434         logging.basicConfig(
435             level=logging.DEBUG if verbose else logging.INFO,
436             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
437             datefmt="%H:%M:%S",
438             force=True,
439         )
440
441
442     def main(argv=None) -> int:
443         args = build_parser().parse_args(argv)
444         _setup_logging(getattr(args, "verbose", False))
445         return args.func(args)
446
447
448     if __name__ == "__main__":
449         sys.exit(main())
450

```

14. assistant (2026-06-21T10:18:47.917Z)

Now let me read both test files.

15. user (2026-06-21T10:18:48.887Z)

```

1         """Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).
2
3         L1 unit coverage of ``backend/config/startup.py``: the fail/warn-fast per-profile
checks.
4         Pure ? no torch/GPU/network (the CUDA probe + env are injected). The cardinal invariant:
5         **no profile selected ? ZERO checks** (default-OFF, identical to pre-M5).
6         """
7         import logging
8
9         import pytest
10
11         from backend.config import AppConfig
12         from backend.config.startup import (
13             LEVEL_ERROR,

```

```

14         LEVEL_WARN,
15         StartupCheckError,
16         enforce_startup_checks,
17         run_startup_checks,
18     )
19
20
21     def _cfg(profile="", backend="local", device="cuda", detector_impl="auto"):
22         """Build an AppConfig with arbitrary profile/storage/device combos ? including ones
23         a
24         preset would never produce (e.g. cloud-serving + local storage), to exercise the
25         coherence
26         checks. model_validate fills every other field with its default."""
27         return AppConfig.model_validate({
28             "deployment": {"profile": profile},
29             "storage": {"backend": backend},
30             "indexing": {"detector_impl": detector_impl, "wasb": {"device": device}},
31         })
32
33     def _codes(checks):
34         return {c.code for c in checks}
35
36     # ----- default-OFF
37
38     @pytest.mark.parametrize("backend,device,detector", [
39         ("local", "cuda", "auto"), ("gcs", "cpu", "stub"), ("s3", "auto", "native"),
40     ])
41     def test_no_profile_is_strict_noop(backend, device, detector):
42         """profile='' ? ZERO checks regardless of any other field (the pre-M5 behaviour)."""
43         cfg = _cfg(profile="", backend=backend, device=device, detector_impl=detector)
44         assert run_startup_checks(cfg, cuda_available=False, env={}) == []
45
46
47     def test_enforce_no_profile_never_raises_and_returns_empty():
48         cfg = _cfg(profile="", backend="local")
49         assert enforce_startup_checks(cfg, cuda_available=False, env={}) == []
50
51
52     # ----- truly-local
53
54     def test_truly_local_local_storage_gpu_present_is_clean():
55         cfg = _cfg(profile="truly-local", backend="local", device="cuda")
56         assert run_startup_checks(cfg, cuda_available=True, env={}) == []
57
58
59     def test_truly_local_cuda_strict_without_gpu_warns():
60         cfg = _cfg(profile="truly-local", backend="local", device="cuda")
61         checks = run_startup_checks(cfg, cuda_available=False, env={})
62         assert _codes(checks) == {"CUDA_NOT_VISIBLE"}
63         assert all(c.level == LEVEL_WARN for c in checks) # never fatal ? healthcheck is
64         the authority
65
66
67     def test_truly_local_auto_without_gpu_warns_cpu_fallback():
68         cfg = _cfg(profile="truly-local", backend="local", device="auto")
69         checks = run_startup_checks(cfg, cuda_available=False, env={})
70         assert _codes(checks) == {"CPU_FALLBACK"}
71
72
73     def test_truly_local_unprobed_cuda_skips_gpu_checks():
74         """cuda_available=None (not probed, e.g. the API server) ? no GPU heads-up at
75         all."""
76         cfg = _cfg(profile="truly-local", backend="local", device="cuda")

```

```

75         assert run_startup_checks(cfg, cuda_available=None, env={}) == []
76
77
78     def test_truly_local_cloud_storage_warns_unusual():
79         cfg = _cfg(profile="truly-local", backend="gcs", device="auto")
80         checks = run_startup_checks(cfg, cuda_available=True, env={})
81         assert "STORAGE_UNUSUAL" in _codes(checks)
82
83
84     def test_truly_local_gdrive_storage_is_fine():
85         cfg = _cfg(profile="truly-local", backend="gdrive", device="cuda")
86         assert run_startup_checks(cfg, cuda_available=True, env={}) == []
87
88
89     # ----- cloud-serving
90
91     def test_cloud_serving_local_storage_is_fatal():
92         cfg = _cfg(profile="cloud-serving", backend="local")
93         checks = run_startup_checks(cfg, cuda_available=True,
env={"GOOGLE_APPLICATION_CREDENTIALS": "/k"})
94         incoherent = [c for c in checks if c.code == "STORAGE_INCOHERENT"]
95         assert incoherent and incoherent[0].level == LEVEL_ERROR
96
97
98     def test_cloud_serving_gcs_without_creds_warns():
99         cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
100         checks = run_startup_checks(cfg, cuda_available=True, env={})
101         assert _codes(checks) == {"CREDS_UNCONFIRMED"}
102         assert all(c.level == LEVEL_WARN for c in checks)
103
104
105     def test_cloud_serving_gcs_with_creds_is_clean():
106         cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
107         assert run_startup_checks(cfg, cuda_available=True,
env={"GOOGLE_APPLICATION_CREDENTIALS": "/k"}) == []
108
109
110
111     def test_cloud_serving_on_cloud_run_env_skips_creds_warn():
112         """K_SERVICE (Cloud Run) implies ADC via the metadata server ? no creds warning."""
113         cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
114         assert run_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "rally"}) ==
[]
115
116
117     def test_cloud_serving_cpu_device_warns():
118         cfg = _cfg(profile="cloud-serving", backend="gcs", device="cpu")
119         checks = run_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "rally"})
120         assert "CLOUD_CPU_DEVICE" in _codes(checks)
121
122
123     def test_cloud_serving_no_gpu_warns():
124         cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
125         checks = run_startup_checks(cfg, cuda_available=False, env={"K_SERVICE": "rally"})
126         assert "CLOUD_CUDA_NOT_VISIBLE" in _codes(checks)
127
128
129     # ----- cpu-mock
130
131     def test_cpu_mock_stub_is_clean():
132         cfg = _cfg(profile="cpu-mock", backend="local", detector_impl="stub")
133         assert run_startup_checks(cfg, cuda_available=False, env={}) == []
134
135
136     def test_cpu_mock_real_detector_warns():
137         cfg = _cfg(profile="cpu-mock", backend="local", detector_impl="native")

```

```

138     checks = run_startup_checks(cfg, cuda_available=False, env={})
139     assert "MOCK_NONSTUB_DETECTOR" in _codes(checks)
140
141
142     # ----- enforce
143
144     def test_enforce_raises_on_fatal_and_carries_failures(caplog):
145         cfg = _cfg(profile="cloud-serving", backend="local")
146         with caplog.at_level(logging.ERROR):
147             with pytest.raises(StartupCheckError) as ei:
148                 enforce_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "x"})
149             assert [c.code for c in ei.value.failures] == ["STORAGE_INCOHERENT"]
150             assert "STORAGE_INCOHERENT" in caplog.text # logged loudly at ERROR before raising
151
152
153     def test_enforce_does_not_raise_on_warnings_only():
154         cfg = _cfg(profile="truly-local", backend="local", device="auto")
155         checks = enforce_startup_checks(cfg, cuda_available=False, env={}) # CPU_FALLBACK
warning
156         assert checks and all(c.level == LEVEL_WARN for c in checks) # returned, not raised
157
158
159     def test_enforce_raise_on_error_false_returns_errors_without_raising():
160         cfg = _cfg(profile="cloud-serving", backend="local")
161         checks = enforce_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "x"},
162                                     raise_on_error=False)
163         assert any(c.code == "STORAGE_INCOHERENT" and c.level == LEVEL_ERROR for c in
checks)
164
165
166     # ----- worker wiring (fail fast before
any work)
167
168     def test_worker_cmd_infer_aborts_rc2_on_incoherent_profile(tmp_path):
169         """The worker's M5 hook aborts cleanly (rc 2) BEFORE any fetch/segmentation when the
active
170         profile is fatally incoherent. The cloud-serving preset sets storage=gcs;
config.json overrides
171         it back to local ? STORAGE_INCOHERENT ? fail fast."""
172         from backend.worker import __main__ as wmain
173
174         cfg = tmp_path / "config.json"
175         cfg.write_text('{"deployment": {"profile": "cloud-serving"}, "storage": {"backend":
"local"}}')
176         video = tmp_path / "in.mp4"
177         video.write_bytes(b"FAKE")
178         rc = wmain.cmd_infer(wmain.build_parser().parse_args([
179             "infer", "--video-uri", str(video), "--out-uri", str(tmp_path / "out"),
180             "--config", str(cfg), "--scratch", str(tmp_path / "s"), "--segmenter",
"wasb_hybrid"]))
181         assert rc == 2
182

```

16. user (2026-06-21T10:18:49.047Z)

```

1     """Profile-parity ? the load-bearing decoupling proof (COMPUTE_DECOUPLED_SERVING M5,
L4).
2
3     The decoupling guarantee: the same `(trajectory + config)` produces the same candidate
4     windows + the same segment ranges** regardless of the active deployment profile /
orchestration.
5     Detection is replaced by the deterministic CPU stub (no GPU/WSL/network), so ANY
divergence
6     is necessarily a profile/seam leak into the pure core ? exactly what D1 forbids.
7

```



```

8      Three arms:
9      * **L4a** ? the WINDOW stage (`trajectory_to_action_windows`) is a pure, profile-
agnostic
10      staticmethod: deterministic across calls + its signature carries no
profile/compute/storage
11      parameter (so it *structurally* cannot branch on a profile).
12      * **L4b** ? the real stub-backed segmenter produces byte-identical segment ranges
under
13      `profile=truly-local` vs `profile=cpu-mock` vs no profile (the only varied knob is
the
14      compute target, which the core never reads).
15      * **L4c** ? the worker (`cmd_infer`, the cloud-serving orchestration) writes an
`intervals.csv`
16      whose ranges match the in-process truly-local run for the same stub trajectory.
17      """
18      import inspect
19      import json
20
21      from backend.config import load_config
22      from backend.infrastructure.database import Database
23      from backend.pipeline.detectors.stub_runner import StubDetectorRunner
24      from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
25
26
27      # A fixed duration so process_video's early ffprobe check passes for the fake
(undecodable) video;
28      # applied identically to every arm so it can never be the source of a divergence.
29      _DURATION = 30.0
30
31
32      def _config(tmp_path, profile):
33          """A config.json applying `profile` (or none), forced to the deterministic CPU stub
+
34          fully-local (no AI handoff) + a tmp output dir. (The stub uses its default synth
params ?
35          `indexing.stub` is not a typed knob, so it deterministically self-configures.)"""
36          data = {
37              "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
38              "output": {"output_dir": str(tmp_path / "out")},
39          }
40          if profile:
41              data["deployment"] = {"profile": profile}
42          cfg_file = tmp_path / f"config_{profile or 'none'}.json"
43          cfg_file.write_text(json.dumps(data))
44          return load_config(cfg_file)
45
46
47      def _ranges(segments):
48          """Normalise a segment list to comparable (start, end) tuples (rounded to drop float
noise)."""
49          out = []
50          for s in segments:
51              start = s.get("start_time", s.get("start"))
52              end = s.get("end_time", s.get("end"))
53              out.append((round(float(start), 3), round(float(end), 3)))
54          return out
55
56
57      def _inprocess_ranges(tmp_path, profile, monkeypatch):
58          tmp_path.mkdir(parents=True, exist_ok=True)
59          # The fake video is undecodable; pin the duration probe so the early validity check
passes.
60          # Applied to the shared module fn ? identical for every profile arm (never a
divergence source).
61

```

```

monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
62         lambda p: _DURATION)
63     db = Database(str(tmp_path / f"db_{profile or 'none'}.sqlite"))
64     cfg = _config(tmp_path, profile)
65     video = tmp_path / "in.mp4"
66     video.write_bytes(b"FAKEVIDEO-PARITY")
67     db.add_video("vidParity", str(video), "processed", []) # FK target for
candidate/play rows
68     seg = WasbHybridSegmenter(db, cfg)
69     result = seg.process_video("vidParity", str(video))
70     return _ranges(result)
71
72
73     # ----- L4a (pure WINDOW
stage)
74
75     def test_window_stage_has_no_profile_parameter():
76         """Structural guard: the WINDOW stage (`trajectory_to_action_windows`) carries no
77         profile/compute/storage parameter, so it *cannot* branch on a deployment profile ?
the
78         precise leak D1 forbids. (Determinism across profiles is exercised end-to-end in
L4b.)"""
79         sig = inspect.signature(StubDetectorRunner.trajectory_to_action_windows)
80         banned = {"profile", "compute_target", "deployment", "storage", "backend"}
81         assert not (banned & set(sig.parameters)), \
82             f"WINDOW stage must not take a profile/compute/storage param; got
{list(sig.parameters)}"
83
84
85     # ----- L4b (in-process
parity)
86
87     def test_inprocess_segment_ranges_identical_across_profiles(tmp_path, monkeypatch):
88         """Same stub trajectory + config under different profiles ? byte-identical segment
ranges.
89         The profiles differ only in compute_target/label, which the pure core never
reads."""
90         local = _inprocess_ranges(tmp_path / "a", "truly-local", monkeypatch)
91         mock = _inprocess_ranges(tmp_path / "b", "cpu-mock", monkeypatch)
92         none = _inprocess_ranges(tmp_path / "c", "", monkeypatch)
93         assert local == mock == none
94         assert len(local) >= 1 # the stub actually produced rallies (the test isn't
vacuous)
95
96
97     # ----- L4c (worker
orchestration)
98
99     def test_worker_intervals_match_inprocess(tmp_path, monkeypatch):
100         """The worker (cmd_infer ? the cloud-serving orchestration) serialises an
intervals.csv whose
101         ranges equal the in-process truly-local run for the same stub trajectory. Proves the
102         fetch/store/serialise orchestration doesn't perturb the core."""
103         from backend.worker import __main__ as wmain
104
105         # In-process reference ranges (this also pins get_video_duration for the worker run
below).
106         ref = _inprocess_ranges(tmp_path / "ref", "truly-local", monkeypatch)
107
108         # Worker path: stub + skip-AI, validators mocked (fake bytes aren't a real video).
109         class _OK:
110             passed = True
111             validator_name = "fake"
112             message = "ok"
113

```

```

114         def model_dump(self):
115             return {"validator_name": "fake", "passed": True, "message": "ok"}
116
117     monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
118                         lambda path, cfg: ([_OK()], {}))
119     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
120
121     cfg_file = tmp_path / "wcfg.json"
122     cfg_file.write_text(json.dumps({
123         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
124         "output": {"output_dir": str(tmp_path / "wout")},
125     }))
126     video = tmp_path / "w.mp4"
127     video.write_bytes(b"FAKEVIDEO-PARITY")
128     out = tmp_path / "bucket"
129
130     rc = wmain.cmd_infer(wmain.build_parser().parse_args([
131         "infer", "--video-uri", str(video), "--out-uri", str(out), "--config",
132         str(cfg_file),
133         "--scratch", str(tmp_path / "scratch"), "--segmenter", "wasb_hybrid"]))
134     assert rc == 0
135
136     # Find the intervals.csv (outputs/<video_id>/intervals.csv).
137     intervals = next(out.rglob("intervals.csv"))
138     rows = intervals.read_text().splitlines()[1:] # drop header
139     worker = [(round(float(r.split(",")[0]), 3), round(float(r.split(",")[1]), 3)) for r
140 in rows]
141     assert worker == ref
142

```

17. assistant (2026-06-21T10:18:59.193Z)

Now I have all the key files. Let me verify several edge cases. First, let me check the config models to understand the getattr-chaining defaults ? specifically whether `deployment`, `storage`, and `indexing.wasb` exist on AppConfig and what happens with a config that has no `deployment` section.

18. user (2026-06-21T10:18:59.757Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720

```

```

27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
71         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72         # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
73         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
74         device: str = "cuda" # auto | cuda | mps | cpu
75         python_bin: str = "python" # the `wasb` conda env python (invoked by path, no
`conda activate`)
76         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
77         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
78         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
79         keep_frames: bool = False
80         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
81         min_free_gb: float = 0.0
82         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,

```

```

83         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
84         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
85         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
86         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
87         # Tri-state:
88         #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89         #                     behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90         #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91         stream_video: Optional[bool] = None
92
93
94     class FusionConfig(BaseModel):
95         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
96
97         Every default reproduces control behaviour: the fusion segmenter persists the
98         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
99         true ? the person-cue features, but it does NOT gate the served handoff unless a
100        threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
101        is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
102        supplied ? off-court persons (spectators / adjacent courts) are excluded.
103        """
104        # Which trajectory substrate seeds the windows (shuttle stays primary).
105        substrate: Literal["wasb", "tracknet"] = "wasb"
106        # Windowing velocity threshold for the fusion family (the engine reads
107        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109        velocity_threshold: float = 0.02
110        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111        # enabled ? so the default path never imports torch / touches the GPU.
112        cue_flags: dict[str, bool] = Field(default_factory=dict)
113        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115        min_crossings: int = Field(default=0, ge=0)
116        # Served Gate B: when the person cue is on, drop windows with median in-court
players
117        # < this. 0 = OFF.
118        min_players: int = Field(default=0, ge=0)
119        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120        court_polygon: Optional[list[list[float]]] = None
121        # Net line for the person two-sided-motion split (person features only ? the shuttle
122        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123        net_axis: Literal["x", "y"] = "y"
124        net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The

```

```

134         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
135         detector_impl: str = "auto"
136         motion_threshold: float = 0.08
137         min_segment_duration: float = 3.0
138         dead_time_merge_gap: float = 2.0
139         chunk_duration: float = 90.0
140         chunk_overlap: float = 10.0
141         detector_model: str = "fasterrcnn_resnet50_fpn"
142         detector_score_threshold: float = 0.5
143         yolo_velocity_threshold: float = 0.15
144         yolo_frame_skip: int = 3
145         yolo_tracker: str = "bytetrack.yaml"
146         yolo_prefetch_workers: int = 2
147         min_action_window_duration: float = 2.0
148         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
149         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
150         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
151         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
152         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
153         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
154         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
155         max_window_duration: float = 6.0
156         window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
157         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
158         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
159         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
160         # Only cuts (recall can't drop). OFF by default until measured/promoted.
161         window_hard_cap: bool = False
162         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
163         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
164         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
165         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
166         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
167         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
168         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
169         window_inactivity_end: float = 1.5
170         inactivity_rally_thresh: float = 0.20
171         inactivity_min_density: float = 0.30
172         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut

```

```

176         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180         window_blob_fallback: bool = False
181         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185         window_blob_factor: float = 4.0
186         log_yolo_candidates: bool = True
187         store_yolo_candidates: bool = True
188         ai_handoff_padding: float = 2.0
189         ai_max_workers: int = 5
190         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
191         # chunks of a high-bitrate (4K) source blow past the provider upload cap
192         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
193         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
194         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
195         # Matches gemini_refine's --clip-scale-width default.
196         ai_chunk_scale_width: int = 1280
197         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
198         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
199         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
200         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
201         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
202         skip_ai_handoff: bool = False
203         # Optional debug knob: trim every video to the first N seconds before processing.
204         debug_duration: Optional[float] = None
205
206         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
207         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208         fusion: FusionConfig = Field(default_factory=FusionConfig)
209
210         @field_validator("default_segmenter")
211         @classmethod
212         def segmenter_must_be_registered(cls, v: str) -> str:
213             # Registry check happens at runtime in main/segmenter selection.
214             # Here we just allow any string for forward compatibility.
215             return v
216
217         @model_validator(mode="after")
218         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
219             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
220             # step makes `while t < duration: t += step` loop forever, growing memory before
any
221             # GPU work. Reject the bad config at load time instead.
222             if self.chunk_overlap >= self.chunk_duration:
223                 raise ValueError(
224                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
225                     f"({self.chunk_duration}); otherwise chunking never advances."
226                 )

```

```

227         return self
228
229
230     class OutputConfig(BaseModel):
231         default_highlights_name: str = "highlights.mp4"
232         db_name: str = "sports_indexer.db"
233         # Directory where the DB, highlight reels, and processed clips are written.
234         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
235         output_dir: str = "output"
236
237
238     class WatermarkConfig(BaseModel):
239         """Branding overlay burned into each highlight clip during the per-clip re-encode
240         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
241         (under `stitching.watermark` in config.json) to turn it off. The text is fully
242         configurable. The concat stage stays stream-copy (-c copy)."""
243         enabled: bool = True
244         text: str = "Generated by Khelsutra.guru"
245         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
246         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
247         font: str = "Sans" # fontconfig family used when font_path is empty
248         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
249         # fraction of frame height
250         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
251         box: bool = True # faint backing box behind the text for legibility
252         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
253         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
254         right"
255
256     class StitchingConfig(BaseModel):
257         begin_padding: float = 0.5
258         end_padding: float = 0.5
259         use_cache: bool = False
260         min_shot_count: int = 1
261         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
262         # duration
263         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
264         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
265         # the
266         # fully-local no-AI path, whereas duration is derived from the rally's own
267         # start/end. The
268         # local detector over-fires and its false positives are mostly SHORT (motion blips,
269         # background-court fragments), so this keeps the reel to the eye-catching long
270         # rallies.
271         # OFF by default ? the detector output is unchanged, only which rallies reach the
272         # reel.
273         min_rally_duration: float = Field(default=0.0, ge=0.0)
274         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
275
276     class LoggingConfig(BaseModel):
277         """Logging knobs for the run endpoints (see backend/utils/logging_setup.py)."""
278         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
279         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
280         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
281         # them to WARNING; set true to restore full chatter for request-flow debugging.
282         verbose_http: bool = False
283
284     class SecurityConfig(BaseModel):
285         # When set, /api/process video_path must resolve under this directory (hosted/tenant
286         # mode). Default None preserves the single-user local 'any local file' workflow.
287         allowed_video_root: Optional[str] = None

```



```

284
285 # --- Chunked upload (B2, PR-2) ---
286 # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287 # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288 # contain upload_dir or /api/process will reject the uploaded paths.
289 upload_dir: str = "output/uploads"
290 # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291 # free-tier edge proxies cap request bodies there (HOSTING_PLAN S2).
292 max_upload_mb: int = 4096
293 max_part_mb: int = 95
294 allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296
297 class StorageConfig(BaseModel):
298     """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
299     behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
300     + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
301     Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
302     **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
303     (boto3/google auth chains supply credentials). See ``backend/storage/``. """
304     backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
305     # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
306     output_root: str = ""
307     # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
308     # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
309     # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
310     # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
311     # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
312     # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
313     # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
314     # scratch dir before processing (see ``backend.storage.acquire_local_input``).
315     input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
316     input_root: str = ""
317     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
318     # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
319     # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
320     # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
321     # the legacy flat layout is used unchanged.
322     single_folder_per_video: bool = False
323     # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
324     workspace_root: str = ""
325     # Cloud namespace prefix under the root so per-video folders sit tidily beside
326     # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
327     workspace_prefix: str = "workspaces"
328     local: dict[str, Any] = Field(default_factory=dict)
329     s3: dict[str, Any] = Field(default_factory=dict)
330     gcs: dict[str, Any] = Field(default_factory=dict)
331     gdrive: dict[str, Any] = Field(default_factory=dict)
332

```

```

333
334 class DeploymentConfig(BaseModel):
335     """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
336     knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
337
338     **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
339     behaves exactly as before this section existed. A profile is opt-in (``config.json``
340     ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
341     *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
342     these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
343     (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
344
345     Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
346
347     # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
348     profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
349     # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
350     # (recorded for observability; consumed by the ComputeBackend seam from M2+).
351     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
352
353
354 class AppConfig(BaseModel):
355     """Root configuration object.
356
357     All runtime configuration should be accessed via an instance of this class
358     (or the loader) rather than raw dict lookups.
359     """
360
361     sport: Sport = "badminton"
362     ai_provider: str = "gemini"
363     # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
364     # the alias never deprecates underneath us. Eval reports record the run date
alongside.
365     ai_model: str = "gemini-flash-latest"
366
367     validation: ValidationConfig = Field(default_factory=ValidationConfig)
368     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
369     output: OutputConfig = Field(default_factory=OutputConfig)
370     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
371     security: SecurityConfig = Field(default_factory=SecurityConfig)
372     logging: LoggingConfig = Field(default_factory=LoggingConfig)
373     storage: StorageConfig = Field(default_factory=StorageConfig)
374     deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
375
376     # Allow extra keys during transition so existing config.json files do not break.
377     model_config = {
378         "extra": "allow",
379         "validate_assignment": True,
380     }
381
382     # Temporary migration shim for legacy dict-style access (config.get("section",
{ }).get(...)).
383     # Will be removed once all call sites (validators, segmenters, etc.) move to typed
384     # attribute access / the typed config. A nested *section* is returned as a plain
dict
385     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
386     # section model instead breaks it (a model has no .get), which bit every validator

```

```

on
387     # real runs. Scalar leaves are returned as-is.
388     def get(self, key: str, default: Any = None) -> Any:
389         _MISSING = object()
390         val: Any = _MISSING
391         if key in type(self).model_fields or hasattr(self, key):
392             val = getattr(self, key, _MISSING)
393         if val is _MISSING:
394             # Legacy convenience: find a leaf key inside any nested section. Sections
are
395             # discovered dynamically from the model fields, so a future top-level
section
396             # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
397             for fname in type(self).model_fields:
398                 section = getattr(self, fname, None)
399                 if isinstance(section, BaseModel) and hasattr(section, key):
400                     val = getattr(section, key)
401                     break
402         if val is _MISSING:
403             return default
404         if isinstance(val, BaseModel):
405             return val.model_dump(mode="json")
406         return val
407
408     def __getitem__(self, key: str) -> Any:
409         val = self.get(key)
410         if val is None and key not in type(self).model_fields:
411             # Re-raise like a real dict for unknown top-level
412             raise KeyError(key)
413         return val
414
415
416     # Convenience type alias for callers that want to annotate
417     Config = AppConfig
418

```

19. assistant (2026-06-21T10:19:08.547Z)

The models always provide `deployment`, `storage`, and `indexing` via `Field(default\_factory=...)`, so getattr-chaining is safe even for a config with no `deployment` section. But the lens specifically asks about a config with "no `deployment` section, no `storage`, no `indexing.wasb`" ? which on a proper AppConfig will never be missing. The defensive `getattr` chains handle the case where someone passes a non-AppConfig (e.g. a dict or legacy object). Let me verify the loader to confirm `--set deployment.profile` would work and confirm the precedence ordering (whether `\_apply\_overrides` correctly runs before the check).

I already confirmed in the worker that `\_apply\_overrides(config, args.set)` runs at lines 110/111 (infer) and 281/282 (train), BEFORE `\_run\_startup\_checks`. Good.

Now let me trace one subtle thing: the `AppConfig.get` migration shim. In `\_run\_startup\_checks`, `getattr(config, "indexing", None)` returns the actual `IndexingConfig` model (since `indexing` is a model field, `getattr` returns the attribute directly, not via the `.get` shim). Let me verify `detector\_impl` default and check whether `\_apply\_overrides` on `deployment.profile` would hit `validate\_assignment`. Let me run the new tests and the broader config/worker suites to confirm green.

20. user (2026-06-21T10:19:19.408Z)

```

..... [100%]
25 passed in 7.46s

```

21. assistant (2026-06-21T10:19:28.072Z)

Both new test files pass. Now let me check a subtle correctness concern: in `main.py`, `enforce\_startup\_checks` is called WITHOUT `cuda\_available`, so it defaults to `None` ? GPU checks skipped (consistent with "server stays torch-light"). Let me verify the placement question ? whether there's any path where `enforce\_startup\_checks` is reached when it should be skipped, or skipped when it should run.

Let me trace `main.py` flow carefully. The early-return branches are: `--setup` (line 526-538), `--trim-start/--trim-end` (540-570), `--debug-clip` (572-577). Each ends with `return`. Then `global\_config = load\_app\_config()` (581), then `enforce\_startup\_checks` (587-591). This means setup/trim/debug never reach the check.

Now a key question: does `--trim` path apply a profile preset and then NOT run the check? Let me think about whether that's a default-OFF violation. The trim path calls `load\_app\_config().stitching` (line 562) ? it loads config (which WOULD apply a profile preset if `deployment.profile` is set in config.json), but never runs startup checks. However, the trim path doesn't do any compute/serving ? it just trims a clip. So skipping the check there is fine (no false-positive, no behavior change). Let me verify there are no OTHER server/worker entrypoints that load config and start serving but bypass the check.

22. user (2026-06-21T10:19:30.116Z)

```
backend\main.py:33:     load_config as load_app_config,
backend\main.py:91:# New code should import load_app_config / AppConfig from backend.config
directly.
backend\main.py:92:def load_config(config_path: str = "config.json") -> Dict[str, Any]:
backend\main.py:93:     cfg = load_app_config(config_path)
backend\main.py:443:     cfg = load_config() # legacy wrapper still works, returns dict for now
backend\main.py:562:         stitching = load_app_config().stitching
backend\main.py:581:     global_config = load_app_config() # returns typed AppConfig
backend\config\loader.py:102:def load_config(path: str | Path | None = None) -> AppConfig:
backend\worker\__main__.py:109:     config = load_config(args.config) if args.config else
load_config()
backend\worker\__main__.py:280:     config = load_config(args.config) if args.config else
load_config()
```

23. user (2026-06-21T10:19:31.081Z)

```
backend\main.py:43:app = FastAPI(title="Sports Highlight Indexer API")
backend\main.py:696:     uvicorn.run(app, host="127.0.0.1", port=args.port)
backend\main.py:698:if __name__ == "__main__":
backend\worker\__main__.py:448:if __name__ == "__main__":
backend\tools\rally_slicer.py:175:if __name__ == "__main__":
backend\eval\calibrate_wasb.py:106:if __name__ == "__main__":
backend\eval\calibrate_local.py:125:if __name__ == "__main__":
backend\eval\ablation.py:492:if __name__ == "__main__":
backend\eval\distill_local.py:164:if __name__ == "__main__":
backend\eval\fusion_audio.py:130:if __name__ == "__main__":
backend\eval\fusion_golden.py:194:if __name__ == "__main__":
backend\eval\eval_partial_labels.py:217:if __name__ == "__main__":
backend\eval\export_wasb_segments.py:229:if __name__ == "__main__":
backend\eval\fusion_compare.py:100:if __name__ == "__main__":
backend\eval\golden_fixtures.py:847:if __name__ == "__main__":
backend\eval\audio\ae1_probe.py:114:if __name__ == "__main__":
backend\eval\gemini_refine.py:232:if __name__ == "__main__":
backend\eval\rally_seg_eval.py:553:if __name__ == "__main__":
backend\eval\rally_seq_proto.py:160:if __name__ == "__main__":
backend\eval\served_gate_a.py:198:if __name__ == "__main__":
backend\pipeline\detectors\wasb_infer.py:715:if __name__ == "__main__":
backend\eval\serve_contrast.py:166:if __name__ == "__main__":
```

24. assistant (2026-06-21T10:19:36.621Z)

The only two production serving/compute entrypoints are `backend/main.py` (server + CLI) and `backend/worker/\_\_main\_\_.py` (worker), both of which now invoke the check. Eval scripts are

offline tooling, out of scope.

Now let me examine one important correctness detail: the `\_apply\_overrides` path and `validate\_assignment`. When the worker does `--set deployment.profile=cloud-serving`, `\_apply\_overrides` walks `config.deployment` then `setattr(deployment, "profile", "cloud-serving")`. Since `validate\_assignment=True` is on `AppConfig` (not necessarily on the nested `DeploymentConfig`), let me verify a `--set deployment.profile` works and is seen by the check. Let me also check the loader to confirm whether `--set deployment.profile` re-applies the preset (it does NOT ? the preset is only applied by the loader at load time, not on `--set`). This is a real subtlety worth checking.

25. user (2026-06-21T10:19:37.110Z)

```
1      """Configuration loader and saver.
2
3      Provides a single place to load validated AppConfig with built-in defaults
4      merged with optional file overrides (config.json).
5
6      This replaces scattered json.load + .get() patterns throughout the codebase.
7      """
8
9      from __future__ import annotations
10
11     import json
12     import logging
13     import os
14     from pathlib import Path
15     from typing import Any
16
17     from pydantic import BaseModel
18
19     from .models import AppConfig
20     from .presets import (
21         COMPUTE_TARGET_FOR_PROFILE,
22         PROFILE_PRESETS,
23         is_known_profile,
24         preset_for,
25         suggest_profile,
26     )
27
28     logger = logging.getLogger(__name__)
29
30     DEFAULT_CONFIG_PATH = Path("config.json")
31
32
33     def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
34         """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
35         scalars
36         replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
37         overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
38         profile
39         path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
40         out = dict(base)
41         for key, val in override.items():
42             if isinstance(val, dict) and isinstance(out.get(key), dict):
43                 out[key] = _deep_merge(out[key], val)
44             else:
45                 out[key] = val
46         return out
47
48     def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
49         """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
50         none
51         is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
```

```

a bare
50     environment never silently selects a profile (and never silently spends on cloud).
51
52     An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
53     profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
54     asserting a preset that was never applied. A bad value in config.json is left to
surface as
55     a hard, typed-Literal error downstream (config-file correctness)."""
56     env_profile = os.environ.get("DEPLOYMENT_PROFILE")
57     if env_profile and env_profile.strip():
58         ep = env_profile.strip()
59         if is_known_profile(ep):
60             return ep
61         logger.warning(
62             "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
63             "and falling back to config.json deployment.profile.", ep, "
".join(PROFILE_PRESETS),
64         )
65         # fall through to the file's profile (env typo must not suppress a valid file
choice)
66     dep = file_data.get("deployment")
67     if isinstance(dep, dict):
68         prof = dep.get("profile")
69         if isinstance(prof, str) and prof.strip():
70             return prof.strip()
71     return ""
72
73
74     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"" ) -> list[str]:
75         """Dotted paths in `data` that the typed config will not honor.
76
77         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
78         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
79         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
80         user's intent is lost ? collect every such key so load_config can warn.
81         """
82         unknown: list[str] = []
83         fields = model_cls.model_fields
84         for key, val in data.items():
85             if key not in fields:
86                 unknown.append(prefix + key)
87                 continue
88             ann = fields[key].annotation
89             if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
90                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
91         return unknown
92
93
94     def _get_builtin_defaults() -> dict[str, Any]:
95         """Return a plain dict of the current built-in defaults.
96
97         The AppConfig model is the single source of truth for defaults.
98         """
99         return AppConfig().model_dump(mode="json")
100
101
102     def load_config(path: str | Path | None = None) -> AppConfig:
103         """Load configuration with the following precedence:
104
105         1. Built-in defaults defined in the Pydantic models.

```

```

106     2. Values from the JSON file (if present and readable).
107     3. (Future) environment / CLI overrides.
108
109     Missing file or unreadable file ? returns a fully defaulted AppConfig.
110     Unknown keys in the file are tolerated (extra="allow" on the model).
111     """
112     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
113
114     file_data: dict[str, Any] = {}
115     if cfg_path.exists():
116         try:
117             with open(cfg_path, "r", encoding="utf-8") as f:
118                 file_data = json.load(f)
119         except Exception as exc:
120             # Non-fatal: continue with defaults + whatever we could parse.
121             # In production we might log here.
122             logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
123
124     # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
`null`) would
125     # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
126     # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
127     # loader's "bad input is non-fatal" contract.
128     if not isinstance(file_data, dict):
129         logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
130                        cfg_path, type(file_data).__name__)
131         file_data = {}
132
133     if file_data:
134         unknown = _unknown_key_paths(file_data, AppConfig)
135         if unknown:
136             logger.warning(
137                 "[CONFIG WARNING] %s contains keys the typed config does not "
138                 "recognize (typo'd or removed knobs ? top-level extras are kept "
139                 "but unread; unknown section keys are silently dropped): %s",
140                 cfg_path, ", ".join(sorted(unknown)),
141             )
142
143     # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
144     # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
145     # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
146     # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
147     defaults = _get_builtin_defaults()
148     profile = _resolve_explicit_profile(file_data)
149
150     if profile and not is_known_profile(profile):
151         # Only an unrecognised value from config.json reaches here (env typos already
fell back
152         # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
153         # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
154         logger.warning(
155             "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
156             profile, ", ".join(PROFILE_PRESETS),
157         )

```

```

158         profile = ""
159
160     if profile:
161         merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
162         # Record the profile actually applied ? the env may have chosen it over
163         config.json,
164         # so re-stamp it onto the merged deployment section to keep the record honest.
165         merged.setdefault("deployment", {})
166         if isinstance(merged["deployment"], dict):
167             merged["deployment"]["profile"] = profile
168             ct = merged["deployment"].get("compute_target")
169             canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
170             if not ct and canonical:
171                 # An active profile with no explicit compute_target (e.g. config.json
172                 # to "") gets its canonical target back ? the record never silently lies
173                 # active profile's compute_target (it stays meaningful for the M2+
174                 ComputeBackend).
175                 merged["deployment"]["compute_target"] = canonical
176                 elif ct and canonical and ct != canonical:
177                     logger.warning(
178                         "[CONFIG] deployment profile %r normally uses compute_target=%r but
179                         "
180                         "config.json overrides it to %r ? honoring the explicit override.",
181                         profile, canonical, ct,
182                     )
183                     logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
184                                 profile, merged["deployment"].get("compute_target"))
185         else:
186             # No profile ? the original shallow merge (Pydantic re-defaults nested
187             sections).
188             merged = {**defaults, **file_data}
189             suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only
190             (D15)
191             if suggestion:
192                 logger.debug(
193                     "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
194                     %r ? "
195                     "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
196                     never "
197                     "auto-applies ? D15).", suggestion,
198                 )
199
200     return AppConfig.model_validate(merged)
201
202
203 def save_default_config(path: str | Path | None = None) -> Path:
204     """Write a fresh default config.json.
205
206     Used by scripts/doctor.py diagnostics and the --setup CLI path.
207     Overwrites if the file exists.
208     """
209     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
210     cfg = AppConfig()
211     cfg_path.parent.mkdir(parents=True, exist_ok=True)
212     cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
213     return cfg_path
214
215
216 # Backwards-compatible alias used by some older call sites during transition.
217 def get_default_config() -> dict[str, Any]:
218     """Return defaults as a plain dict (for transitional use only)."""
219     return _get_builtin_defaults()
220
221

```


26. assistant (2026-06-21T10:19:44.784Z)

Now I understand a real subtlety worth checking. When a user runs the worker with `--set deployment.profile=cloud-serving`` (rather than putting it in `config.json`), the loader applies NO preset (the preset is only applied at load time based on `config.json/env`). So `_apply_overrides`` sets `deployment.profile="cloud-serving"` but leaves `storage.backend="local"` (the default). The check would then fire `STORAGE_INCOHERENT` ? fatal ? rc 2`.

Is that a false-positive? In this scenario the user explicitly set `--set deployment.profile=cloud-serving`` but didn't set storage. The preset that would've set `storage.backend=gcs`` was NOT applied (since `--set`` is post-load). So the check correctly flags that the config is genuinely incoherent (cloud profile + local storage). This is arguably *\*intended\** behavior ? but it's a subtle trap: a user might reasonably expect `--set deployment.profile=cloud-serving`` to behave like the preset. However, this matches the documented precedence (preset applied only by loader for `config.json/env` profile, `--set`` is a raw override). Not a bug ? and not a default-OFF violation (default profile is still ""). I'll note this as a minor observation but it's actually correct behavior given the documented contract.

Let me now verify the precedence test concern with `validate_assignment`` ? does setting `deployment.profile`` via `_apply_overrides`` actually persist? Let me run a quick check.

27. user (2026-06-21T10:19:53.868Z)

```
before profile= '' storage= local
after  profile= 'cloud-serving' storage= local
checks: [('error', 'STORAGE_INCOHERENT')]
```

28. assistant (2026-06-21T10:19:59.335Z)

Confirmed: `--set deployment.profile`` is correctly seen by the check (the `_apply_overrides`` runs before `_run_startup_checks``).

Now let me verify the critical default-OFF invariant for `main.py`` server path ? that with `profile=""``, `enforce_startup_checks`` is a true no-op (logs nothing, raises nothing). Let me trace it once more and verify with an actual run that no log line is emitted.

29. user (2026-06-21T10:20:02.859Z)

```
returned: []
log output (should be empty): ''
```

30. assistant (2026-06-21T10:20:07.673Z)

Confirmed strict no-op: empty profile returns `[]``, logs nothing, raises nothing.

Now let me check the edge case the lens raised about a config "with no `indexing.wasb``" causing `device`` resolution issues. On a real `AppConfig``, `indexing.wasb`` always exists. But the `getattr(getattr(indexing, "wasb", None), "device", "cuda")`` chain: if `indexing`` is `None`` (non-`AppConfig``), `getattr(None, "wasb", None)`` ? `None``, then `getattr(None, "device", "cuda")`` ? `"cuda"``. Safe. Let me verify there's no `AttributeError` when passing a bare object with no sections.

31. user (2026-06-21T10:20:12.734Z)

```
bare, no profile: []
cloud-serving, no storage/indexing: [('error', 'STORAGE_INCOHERENT'), ('warn',
'CLoud_CUDA_NOT_VISIBLE')]
truly-local bare: [('warn', 'CUDA_NOT_VISIBLE')]
```

32. assistant (2026-06-21T10:20:22.167Z)

No `AttributeError` in any of these degenerate cases. The `getattr`-chaining defaults safely: missing storage ? `"local"`, missing indexing ? device `"cuda"`.

Note: for the bare cloud-serving object, `storage\_backend` defaults to `"local"` ?
STORAGE_INCOHERENT fires. That's the correct semantics (a config with no storage means local).
Good.

Now let me verify the worker abort vs main.py exit code question. Worker returns `2` from
`cmd\_infer`/`cmd\_train`, which `main()` returns and `sys.exit(main())` propagates ? exit code 2.
main.py server uses `sys.exit(1)`. Both are non-zero/fatal. Different codes but both correct for
their context (worker has a documented rc convention: 2 = pre-flight/validation failure, 3 =
segmenter failure). Let me confirm rc 2 is consistent with the worker's existing convention.

Looking at the worker: validation failure ? `return 2` (line 141), unknown segmenter ? `return
2` (line 147), `<2 labels` ? `return 2` (line 299). So rc 2 = "pre-flight / input-coherence
failure" is the established convention, and the M5 startup-check fatal correctly reuses it. The
main.py `sys.exit(1)` is the standard generic-fatal for a server that refuses to start. Both
correct.

Let me now run the broader config + worker test suites to confirm no regression, and confirm the
byte-identical default-OFF claim holds at the loader level (no profile ? unchanged merge).

33. user (2026-06-21T10:20:24.890Z)

ERROR: file or directory not found: tests/test_config_loader.py

no tests ran in 0.00s

34. user (2026-06-21T10:20:29.594Z)

test_config.py
test_config_deployment.py
test_config_wasb.py
test_job_worker_loop.py
test_job_worker_module_seam.py
test_profile_parity.py
test_sport_profiles.py
test_startup_checks.py
test_worker.py

35. user (2026-06-21T10:20:41.979Z)

..... [69%]
..... [100%]
104 passed in 7.31s

36. assistant (2026-06-21T10:20:50.051Z)

All 104 pass. Now let me check the RUNLOG template that was added and verify the worker's
`main.py` server path doesn't have a subtle issue: the `enforce\_startup\_checks` in main.py
passes `logger=logger` but no `cuda\_available` ? for `truly-local`/`cloud-serving` profiles
selected via config.json on the SERVER, GPU warnings are skipped (cuda_available=None). That's
the documented intent (server is torch-light). But is there a scenario where the server SHOULD
warn but doesn't? The docstring says the detector healthcheck is the GPU authority, so skipping
GPU warnings on the server is deliberate and fine.

Let me verify one more correctness angle: the test
`test\_worker\_cmd\_infer\_aborts\_rc2\_on\_incoherent\_profile` uses config.json with `cloud-serving` +
`storage.backend=local`. Per the loader, when config.json has `deployment.profile=cloud-
serving`, the preset WOULD set `storage.backend=gcs`, but config.json explicitly overrides it
back to `local`. Let me confirm the loader's deep-merge produces `local` (config.json wins over
preset) so the test scenario is real.

37. user (2026-06-21T10:20:52.881Z)

Exit code 1

```
Traceback (most recent call last):
  File "<string>", line 2, in <module>
    from backend.config import load_config
ModuleNotFoundError: No module named 'backend'
```

38. assistant (2026-06-21T10:20:55.112Z)

The ``cd /tmp`` broke the module path. Let me run from the repo root.

39. user (2026-06-21T10:20:57.567Z)

```
profile= storage= local compute_target=
checks: []
```

40. assistant (2026-06-21T10:21:02.936Z)

Interesting ? this reveals a subtle but important finding. When ``config.json`` sets ``deployment.profile=cloud-serving`` AND ``storage.backend=local``, the loaded config has ``profile=""`` (empty!) and ``storage=local``, so the check is a no-op.

Wait ? that's unexpected. The loader should have applied ``cloud-serving``. Let me investigate why the profile came back empty. The deep-merge: ``_deep_merge(_deep_merge(defaults, preset_for("cloud-serving")), file_data)``. The preset sets ``deployment.profile=cloud-serving``, then `file_data` has ``deployment={profile: cloud-serving}`` too. Then line 166 re-stamps ``merged["deployment"]["profile"] = profile``. So profile should be ``cloud-serving``. But it came back ``""``. Let me dig ? maybe ``DEPLOYMENT_PROFILE`` env isn't set and something is wrong. Let me add debug.

41. user (2026-06-21T10:21:05.185Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 6, in <module>
    fd = json.load(open('/tmp/m5cfg.json'))
    ~~~~^~~~~~
FileNotFoundError: [Errno 2] No such file or directory: '/tmp/m5cfg.json'
```

42. assistant (2026-06-21T10:21:09.529Z)

The ``/tmp`` path differs between Bash (Git Bash) and the Python process (Windows). Let me use a Windows-friendly path.

43. user (2026-06-21T10:21:13.138Z)

```
INFO:backend.config.loader:[CONFIG] deployment profile ACTIVE: cloud-serving
(compute_target=cloud_run)
file_data= {'deployment': {'profile': 'cloud-serving'}, 'storage': {'backend': 'local'}}
resolved profile= 'cloud-serving'
loaded profile= 'cloud-serving' storage= local ct= 'cloud_run'
checks: [('error', 'STORAGE_INCOHERENT')]
```

44. assistant (2026-06-21T10:21:18.856Z)

That confirms it. My earlier "empty profile" result was a path artifact: the Git Bash ``/tmp/m5cfg.json`` was never visible to the Windows Python process, so ``load_config('/tmp/m5cfg.json')`` saw a non-existent file ? defaulted to ``profile=""``. With a real Windows temp path, the loader correctly applies ``cloud-serving``, keeps ``storage=local`` (config.json wins), and the check fires ``STORAGE_INCOHERENT`` (error) as intended. The test scenario is real and correct. No bug ? that was my test harness path mismatch, not a code issue.

Now let me look at the RUNLOG template and confirm it's just docs (no behavioral impact), and do a final check on whether anything in the diff touches a default code path.

45. user (2026-06-21T10:21:20.224Z)

```
diff --git a/backend/config/__init__.py b/backend/config/__init__.py
index 493c535..9b276c0 100644
--- a/backend/config/__init__.py
+++ b/backend/config/__init__.py
@@ -16,7 +16,13 @@ from .models import ( # re-export common ones
    StorageConfig,
    ValidationConfig,
)
-from .presets import PROFILE_PRESETS, suggest_profile
+from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
+from .startup import (
+    StartupCheck,
+    StartupCheckError,
+    enforce_startup_checks,
+    run_startup_checks,
+)

__all__ = [
    "load_config",
@@ -30,4 +36,9 @@ __all__ = [
    "ValidationConfig",
    "PROFILE_PRESETS",
    "suggest_profile",
+    "probe_cuda_available",
+    "StartupCheck",
+    "StartupCheckError",
+    "enforce_startup_checks",
+    "run_startup_checks",
]
diff --git a/docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_truly-local_TEMPLATE.md
b/docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_truly-local_TEMPLATE.md
new file mode 100644
index 0000000..84c080a
--- /dev/null
+++ b/docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_truly-local_TEMPLATE.md
@@ -0,0 +1,49 @@
+# Runlog ? truly-local profile (TEMPLATE ? owner fills on a real GPU run)
+
+> **Status:** TEMPLATE (pre-staged by M5 so the owner's L5 run is one-shot). Copy to
+> `RUNLOG_truly-local_<YYYY-MM-DD>.md`, fill the blanks from a real run, commit. This satisfies
the
+> §9 DoD "truly-local has ?1 committed sample run" and the TESTING_STRATEGY L5 owner-side gate.
+> **Profile:** `truly-local` (`compute_target=local_gpu`, `storage=local`, stitch in-process,
zero cloud).
+
+### 0. Environment
+- Box / GPU: `_____` (e.g. Windows + RTX 2070 via WSL, or a Linux GPU box)
+- `detector_impl`: `_____` (`auto` = WSL runner / `native` = Linux-native WASB)
+- `indexing.wasb.device`: `_____` (`cuda` strict, or `auto` for the M4 CPU fallback)
+- torch / CUDA: `_____` (`torch.cuda.is_available()` = `_____`)
+- Input video(s): `_____` (golden-corpus clip name + resolution + fps)
+
+### 1. Exact commands
+```bash
+# Select the profile + run truly-local end-to-end (detect ? window ? stitch in-process).
+export DEPLOYMENT_PROFILE=truly-local
+# (server) python -m backend.main --server      # or
+# (CLI)    python -m backend.main --video-path <clip> --segmenter wasb_hybrid
+# (worker) python -m backend.worker infer --video-uri <clip> --out-uri <local-out> --segmenter
wasb_hybrid
+```
+- Startup-check output (M5): paste the `[STARTUP] ?` lines (expect none, or a CPU_FALLBACK/CUDA
+ warning if device=auto on a no-GPU box). A clean truly-local run prints **no STARTUP
```

```

errors**.
+
+## 2. Results
+- Rallies detected: `____` (segment count)
+- A few segment ranges (start?end): `____`
+- Reel produced: `____` (path) · size `____`
+- `gpu_active_seconds` / `wall_seconds` (if measured): `____` / `____`
+
+## 3. Reel smoke-test (TESTING_STRATEGY $5)
+```bash
+ffmpeg -v error -i <reel.mp4> -f null -          # full decode ? exit 0 (no corruption)
+# 5-frame contact sheet to eyeball real rallies:
+ffmpeg -i <reel.mp4> -vf "fps=5/<reel_seconds>,scale=320:-1,tile=5x1" contact_sheet.png
+```
+- `ffmpeg -f null -` exit code: `____` (expect 0)
+- Contact sheet attached / looks like real rallies: `____`
+
+## 4. Parity sanity (optional, $0)
+- The L4 profile-parity test (`tests/test_profile_parity.py`) already proves windows+segment
+ ranges are byte-identical across profiles for a fixed (stub) trajectory. On a real run, note
+ whether the rally COUNT is in the expected ballpark for the clip (rally-level, not byte-level
+ ?
+ cross-GPU detection is sub-pixel-different by design, D5).
+
+## 5. Gotchas / teardown
+- Notes: `____` (conda-ToS, WASB env, font fallback, Modern-Standby keepawake, etc.)
+- Teardown / disk: `____` (truly-local has nothing to bill; just cache-hygiene)

```

46. assistant (2026-06-21T10:21:24.392Z)

Both are non-behavioral: `\_\_init\_\_.py` is pure additive exports (no existing import changed), and the runlog is a docs template. No default code path touched.

Let me do a final verification: confirm the diff to `presets.py` is purely additive (the new `probe\_cuda\_available` wrapping the existing `\_cuda\_available`), and that nothing in the existing `suggest\_profile`/loader path changed behavior.

47. user (2026-06-21T10:21:25.637Z)

```

diff --git a/backend/config/presets.py b/backend/config/presets.py
index f793227..5d49333 100644
--- a/backend/config/presets.py
+++ b/backend/config/presets.py
@@ -94,6 +94,18 @@ def _cuda_available() -> bool:
     return False

+def probe_cuda_available() -> bool:
+    """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
+
+    Exposed for the per-profile startup checks (`backend/config/startup.py`) so a caller
+    that *wants* the GPU heads-up can pass the result in without the startup module importing
+    torch itself. Best-effort by design: if torch lives only in a detector subprocess env
+    (native WASB), this returns ``False`` in the main process ? which is why the startup checks
+    treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a hard
+    error that could false-fail."""
+    return _cuda_available()
+
+def suggest_profile(
+    env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
+) -> str:
diff --git a/backend/main.py b/backend/main.py
index 75888d3..8e05ce9 100644
--- a/backend/main.py

```

```

+++ b/backend/main.py
@@ -29,7 +29,13 @@ from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler
    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
    from backend.utils.paths import resolve_under_root, safe_output_filename, validate_input_path
-from backend.config import load_config as load_app_config, AppConfig, StitchingConfig # new
typed config
+from backend.config import ( # new typed config
+    load_config as load_app_config,
+    AppConfig,
+    StitchingConfig,
+    enforce_startup_checks,
+    StartupCheckError,
+)
    from backend.results import Failure # standardized error/result contract
    from backend.reporting import emit_indexer_report
    from backend import storage # M2: URI-aware input acquisition (local = identity passthrough)
@@ -574,6 +580,16 @@ def main():
    global global_config, db_instance
    global_config = load_app_config() # returns typed AppConfig

+    # M5: per-profile startup checks ? fail fast on an incoherent ACTIVE profile (e.g. a
+    # cloud-serving profile pointed at local-only storage) BEFORE any work. A strict no-op when
+    # no deployment.profile is selected (the default), so today's behaviour is unchanged. No
GPU
+    # probe here (the server stays torch-light; the detector healthcheck is the GPU authority).
+    try:
+        enforce_startup_checks(global_config, logger=logger)
+    except StartupCheckError as e:
+        logger.error("[STARTUP] refusing to start: %s", e)
+        sys.exit(1)
+
    # The CLI --output-dir overrides the configured output directory. It now lives
    # on the typed model (OutputConfig.output_dir), so every consumer ? including
    # /api/compile ? reads the same value instead of a write-only runtime hack.
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index fafb35a..c644ede 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -25,7 +25,12 @@ import tempfile
    from typing import Any, List

    from backend import storage
-from backend.config import load_config
+from backend.config import (
+    StartupCheckError,
+    enforce_startup_checks,
+    load_config,
+    probe_cuda_available,
+)
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.validators.base import registry as validator_registry
@@ -88,9 +93,23 @@ def _write_report_json(video_id: str, segments: List[dict], status: str,
path: str
    }, f, indent=2)

+def _run_startup_checks(config) -> bool:
+    """M5 per-profile startup checks for the worker (the compute process, so it probes CUDA
+    best-effort). Logs warnings; returns False on a FATAL coherence error so the caller aborts
+    cleanly (rc 2) rather than run an expensive job under an incoherent profile. A no-op
+    (returns
+    + True) when no deployment.profile is selected."""
+    try:

```

```

+         enforce_startup_checks(config, cuda_available=probe_cuda_available(), logger=logger)
+         return True
+     except StartupCheckError:
+         return False # already logged at ERROR by enforce_startup_checks
+
+
+ def cmd_infer(args: argparse.Namespace) -> int:
+     config = load_config(args.config) if args.config else load_config()
+     _apply_overrides(config, args.set)
+     if not _run_startup_checks(config):
+         return 2
+     storage_cfg = config.storage.model_dump()
+
+     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
@@ -260,6 +279,8 @@ def cmd_train(args: argparse.Namespace) -> int:
+
+     config = load_config(args.config) if args.config else load_config()
+     _apply_overrides(config, args.set)
+     if not _run_startup_checks(config):
+         return 2
+     storage_cfg = config.storage.model_dump()
+
+     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")

```

48. assistant (2026-06-21T10:21:32.523Z)

The full diff confirms the change is tightly scoped and additive. Let me verify one last thing ? the placement of the check in `main.py` relative to the `--output-dir` override. The check runs at line 587 (right after `load\_app\_config()`), BEFORE `global\_config.output.output\_dir = args.output\_dir` (line 596). This means the startup check sees the config as loaded, not with the `--output-dir` override applied. Since none of the M5 checks read `output.output\_dir`, this ordering is irrelevant to correctness. Good.

One more: the worker's `cmd\_train` with `--trajectory-source detector` does heavy GPU work; the check at line 282 runs before `\_resolve\_train\_detector`, so it fails fast as intended. And `cmd\_infer` check at line 111 runs before `storage.fetch` (line 120), so it fails fast before any download. Both correctly placed.

I have now traced every path the lens asked about. Let me compile my findings. The implementation is correct and the default-OFF contract holds strictly. I found no real bugs ? only a couple of design observations I considered and confirmed are intentional/acceptable.

49. user (2026-06-21T10:21:47.500Z)

Structured output provided successfully

50. assistant (2026-06-21T10:22:00.519Z)

I have completed the CORRECTNESS & DEFAULT-OFF SAFETY review of the M5 change on `feat/serving-m5-truly-local-checks`.

Summary of what I verified (all holding):

1. `**profile=""` is a STRICT no-op everywhere ? traced and confirmed live: `run\_startup\_checks` returns `[]` at `startup.py:80-81` before reading any other field; `enforce\_startup\_checks` then logs nothing and raises nothing (verified empirically ? empty profile yields `returned: []`, log output `''`). The L1 test `test\_no\_profile\_is\_strict\_noop` pins this across every storage/device/detector combo.
2. `**enforce_startup_checks` placement in main.py:588 is correct**` ? it sits after the early-return CLI branches (`--setup`/`--trim`/`--debug-clip`, each ending in `return`) and after `load_app_config()`, so those non-compute paths never trigger it, and before the server+CLI-process work. It reads no `output.output_dir`, so running it before the `--output-dir` override is irrelevant. The only two production serving/compute entrypoints (`backend/main.py`,

`backend/worker/`__main__.py`) both invoke the check; eval scripts are offline tooling.

3. ****getattr-chaining defaults safely**** ? verified with bare objects having no
`deployment`/`storage`/`indexing`/`wasb` sections: no `AttributeError`; missing storage defaults
to `"local"`, missing device to `"cuda"`. On a real `AppConfig` these sections always exist via
`Field(default\_factory=...)`.

4. ****Worker rc 2 vs main.py sys.exit(1)**** ? both correct. rc 2 is the worker's established pre-
flight-failure convention (validation failure, unknown segmenter, `<<2 labels` all return 2);
`sys.exit(1)` is the standard server-refuses-to-start code.

5. ****`\_apply\_overrides` runs BEFORE the check**** ? confirmed at `\_\_main\_\_.py:110-111` (cmd_infer)
and `281-282` (cmd_train), and verified live that `--set deployment.profile=cloud-serving` is
seen by `run\_startup\_checks`.

Tests: the two new files pass (25), and the broader config/worker/deployment suite (104 total)
is green.

I found ****no blockers, majors, or minors**** ? only two `nit`-level design observations that I
confirmed are intentional and acceptable (the `--set` vs config.json preset asymmetry, and the
deliberate server-side GPU-warning skip). Relevant files: `C:\Users\avidu\Projects\badminton-
highlight-indexer\backend\config\startup.py`, `backend\main.py`, `backend\worker__main__.py`,
`backend\config\loader.py`.